Session 1: how to run it

Session 1 opens on Scribble, Pencil's AI agent, takes it apart, and builds it back one part at a time, while participants build LaunchKit, a mini Pencil. This guide walks the trainer through the session from the opening to the ending: the story line, the live moments, the hand-offs and the discussion points for each step.

How the session is told: Take Scribble apart. Build it back.

The session opens on Scribble as it works today, takes it apart, and builds it back one part at a time, starting from Maya's simplest problem. Each step adds one part, brought in because Maya needs it and because the models of the time could do only so much. No approach is told as bad: many changes came because the models got stronger, not because the old way failed. Each step also gives the trainer discussion points about other ways to solve it.

Scribble's eight parts

The story is built around Scribble's eight parts, as its docs name them: Model, Experts, Lookup, DNA, Main agent, Skills, Checks and Brand. Maya's brief is not a part; it comes from outside. Step 0 introduces all eight, each step after it builds one back, and step 8 walks all eight again, with three of them (DNA, Checks and Brand) built by the room.

The opening

The opening explains Pencil in plain words, then hands the room Maya's problem. It has five beats, one of them the team's platform video.

No customer is named. After the beats, introduce Scribble as a whole, then set it aside to build it back from nothing.

Beat	What it says	Link to the hands-on
1. One idea, hundreds of ads	A shampoo brand sells to students, parents and athletes, and TikTok, Instagram and YouTube each need the ad in a different size. One campaign can have 12 ideas and need 200+ ads. People used to make them by hand.	Maya has the same problem, only smaller.
2. What Pencil is	One place where a brand's whole team makes ads with AI, from brief to launch. A marketer types a brief and gets ideas, pictures and video ads. One brief becomes thousands of versions, in other languages and markets. People fix them by hand in the editor, with the brand's logos and photos one click away. Legal and brand teams approve them in the same place. Pencil publishes to Meta, Google and TikTok, and predicts which ads will do best. It uses models from Anthropic, Google, Adobe and others, checks every ad against the brand's rules, and never trains AI on a customer's data.	LaunchKit copies one slice: a brief becomes a concept, storyboard, copy, poster and every size, checked against the brand's rules, with each brand kept apart. Editing, approval, launch and prediction stay out; the ending points back to them. "Many models" comes back at the move to Agno.

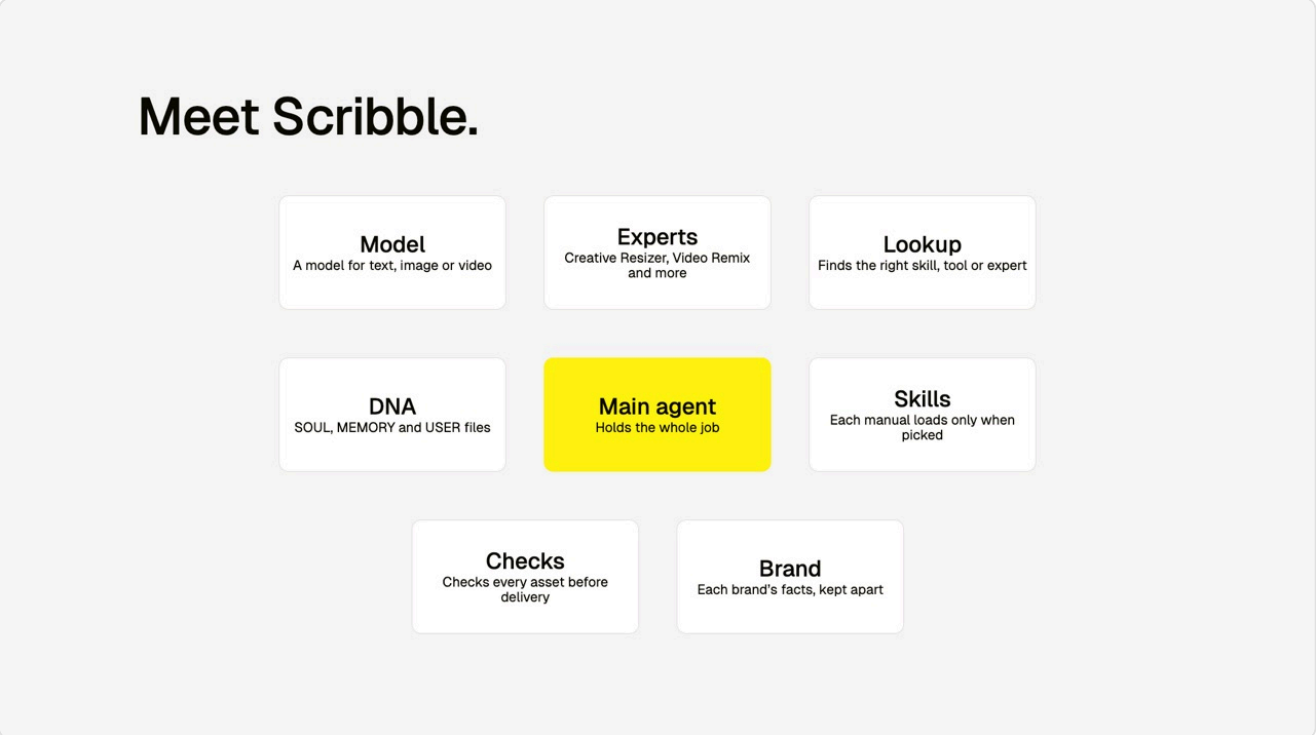
3. The recording	The team's short platform video.	Watch the video
4. People and agents work together	Teams still make and finish ads by hand in Pencil's editor. AI agents help across the whole job: they research the audience and the market, check and write briefs, make pictures, video, voice and storyboards, adapt them to every format, and check and score the results. Pencil has had agents since its ADK days; today Scribble reads a plain request and looks up the right expert, skill or tool. Each brand has its own workspace.	We copy the agent side only. The ending sends LaunchKit's poster to an editor for a person to finish.
5. Meet Maya	Maya is Harbor's marketer. Harbor makes insulated bottles. Tidal launches in six weeks, and she needs about 40 posters. She asks: "Why don't I just use ChatGPT?"	The question stays open until the ending.

Step by step

Each step below tells one part of the story as Maya lives it: the problem she hits, the solution that adds one part of Scribble, and the issue that pushes the story into the next step. Live moments and trainer discussion points sit where they happen in the room.

Step 0 · Pencil today

The session starts with what Pencil does. One idea becomes hundreds of ads, and Pencil is one place to make them with AI, where people and AI agents work together. Today one AI agent, Scribble, does much of the agent work. Run the opening beats (above), then introduce Scribble's eight parts one by one, without explaining any of them yet.

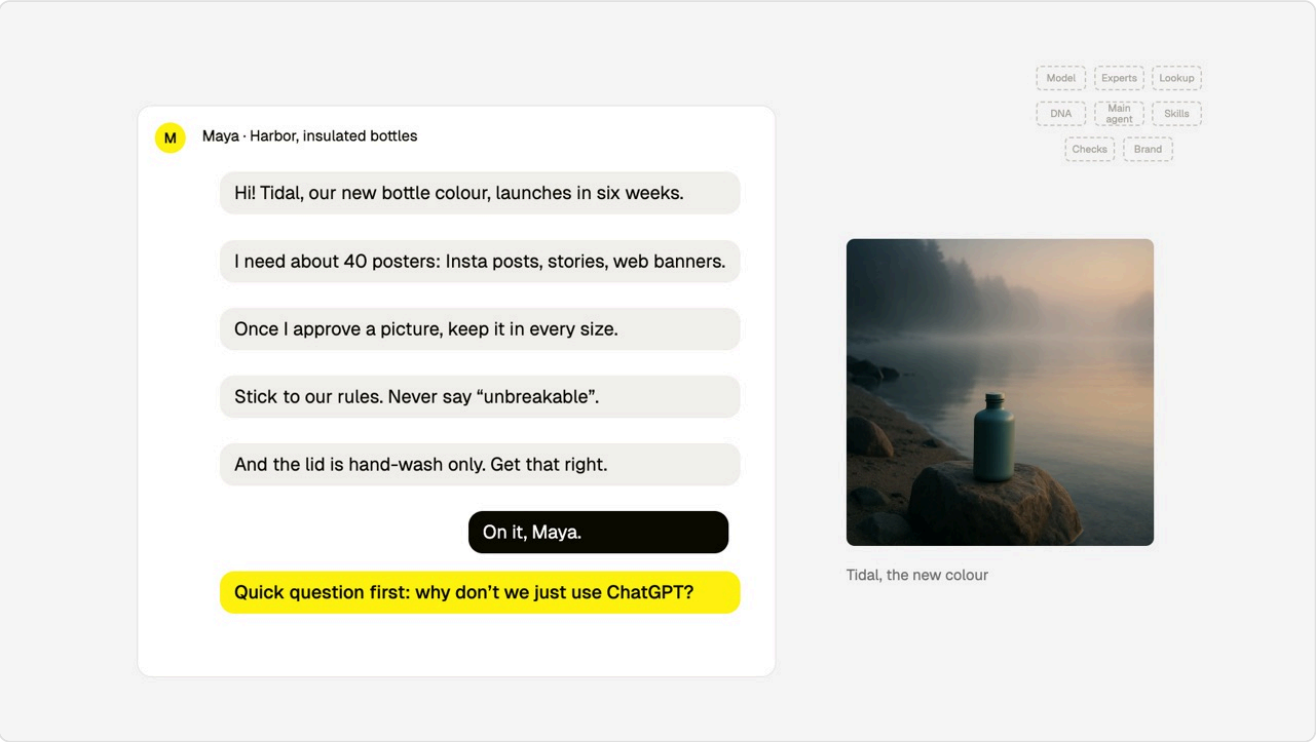


In the room. Before Scribble's parts come up, ask the room what parts an AI ad maker needs. Then compare their list with Scribble's.

Into step 1. "Why does it look like this? Let's take Scribble apart and build it back." The story starts again from nothing: one marketer and her problem.

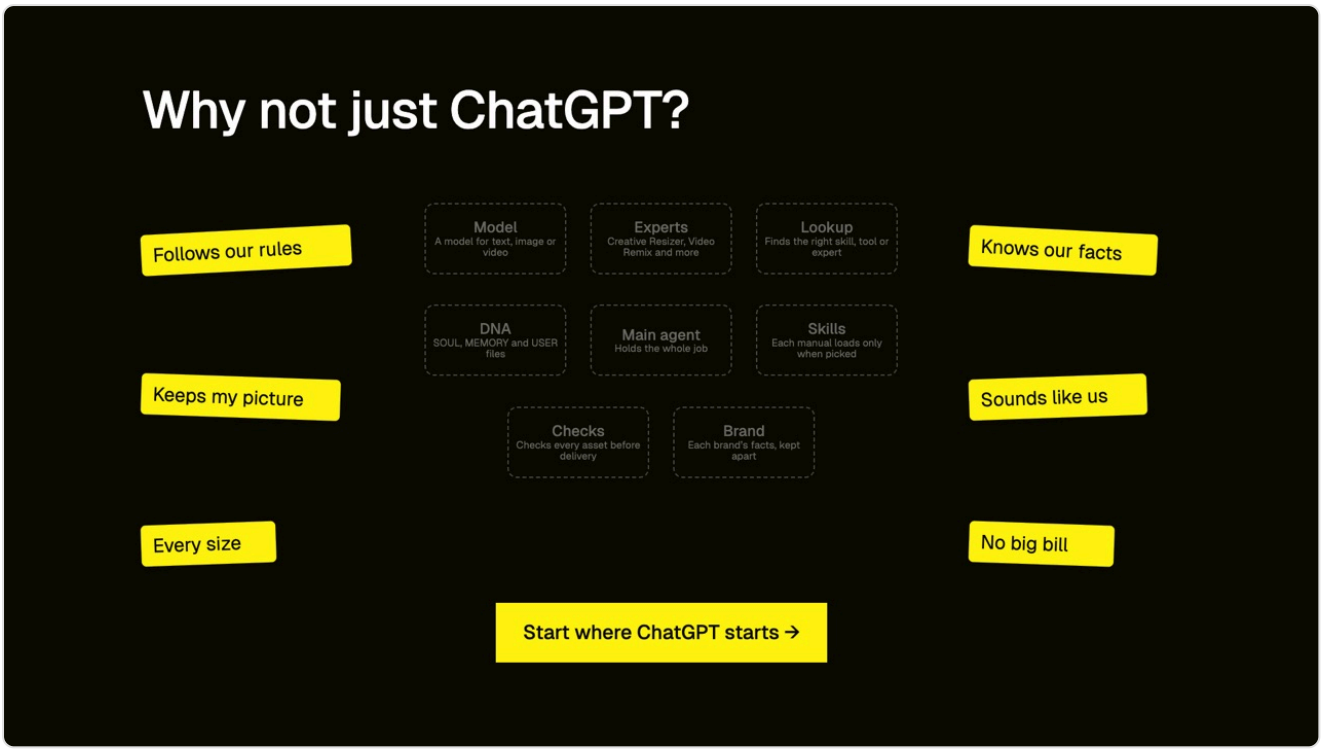
Step 1 · Maya's problem: why not just use ChatGPT?

The problem. Maya, Harbor's marketer, chats with the team in her own words. Tidal launches in six weeks and she needs about 40 posters for Insta posts, stories and web banners. The approved picture must stay the same in every size, the word "unbreakable" is banned, and the lid is hand-wash only. Her last line: "Quick question first: why don't we just use ChatGPT?"



Maya's brief is not a part of Scribble: it comes from outside.

In the room. Take the room's answers to Maya's question first and write them down. Then match each of Maya's needs to the part of Scribble that answers it: follows our rules and no big bill → Checks, keeps my picture → Main agent, every size → Skills, knows our facts → Brand, sounds like us → DNA. Step 8 comes back to this list and closes each one.



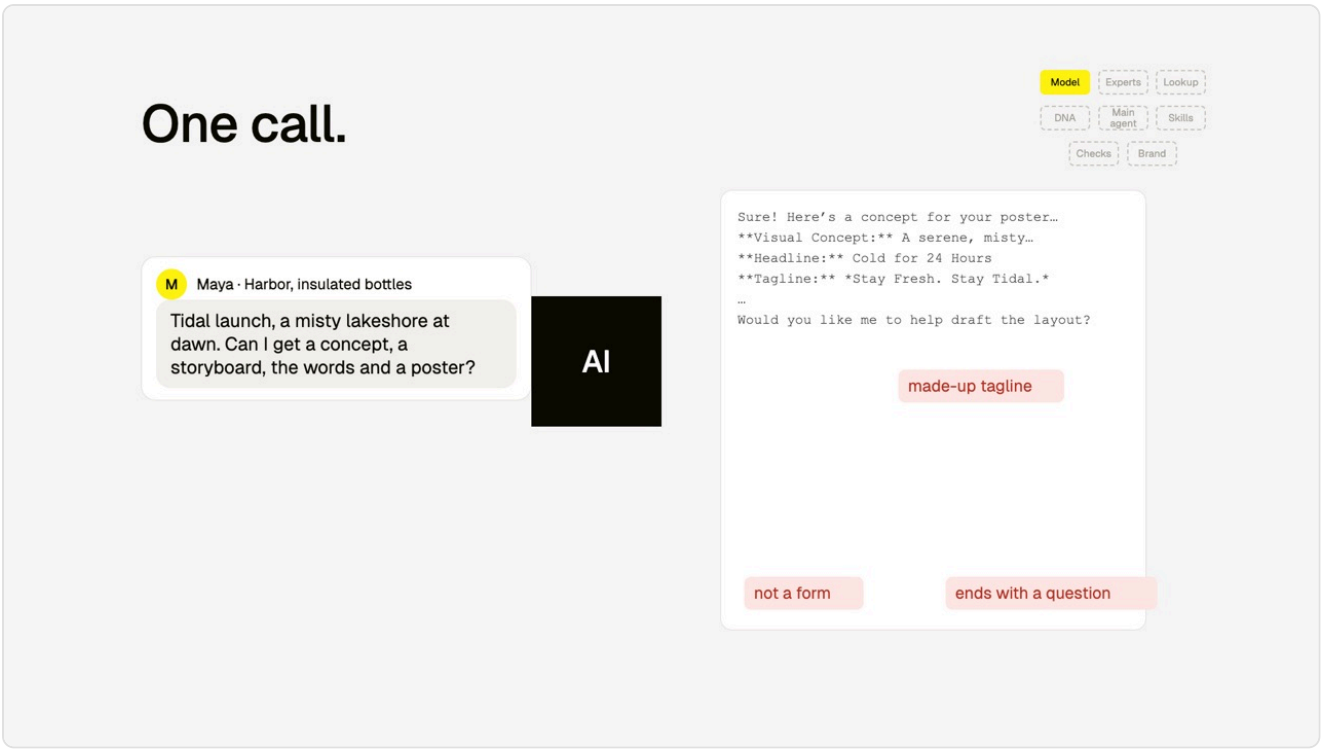
Keep it fair. Don't put ChatGPT down: it is very good at answering one person in a chat. Maya needs a product with rules, limits and brands built in.

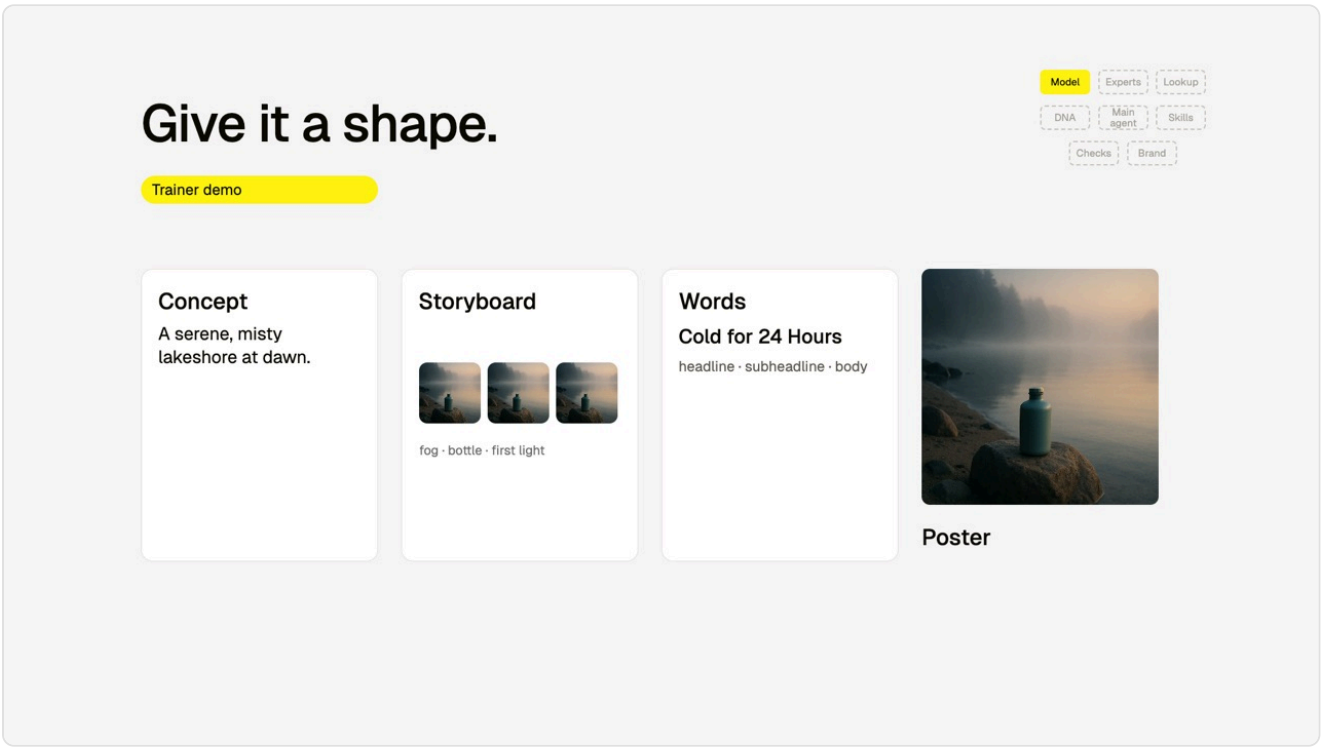
Into step 2. So we start where ChatGPT starts: one call to a model.

Step 2 · One AI call

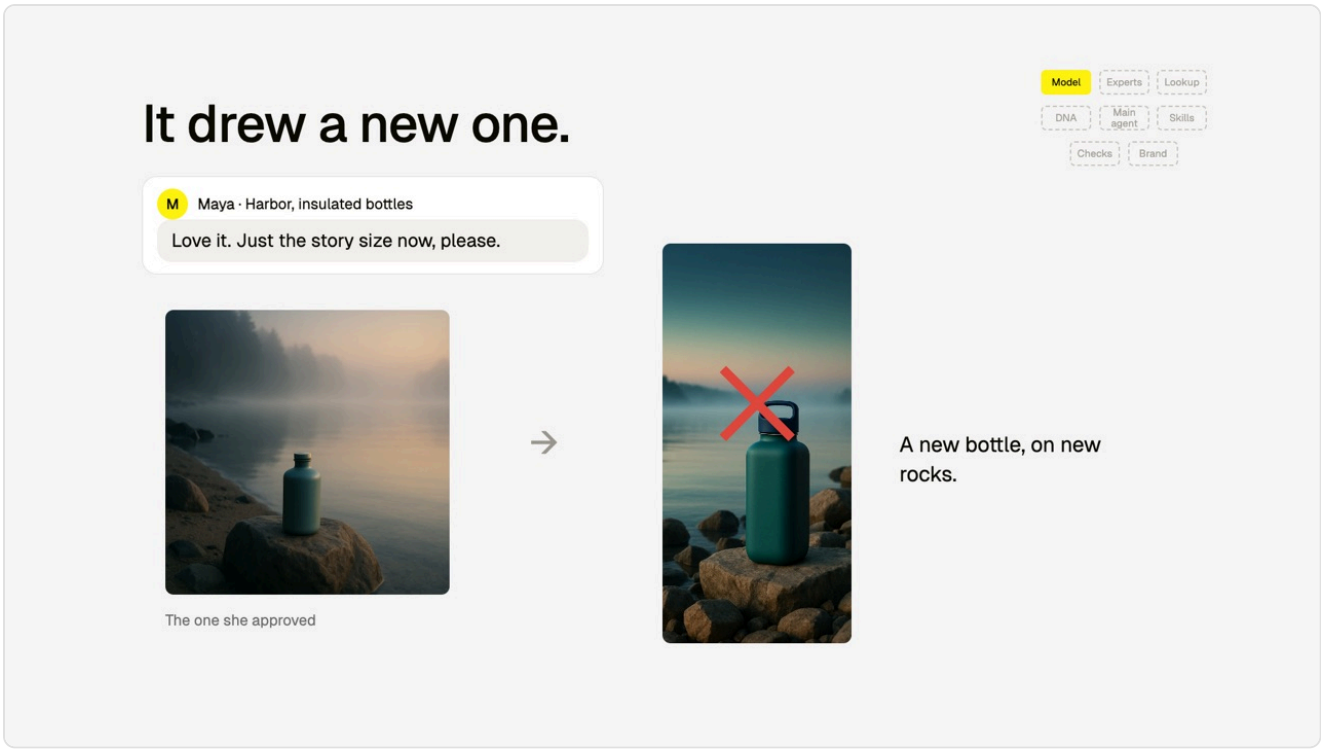
The solution. Our engineer sends Maya's whole brief to the AI in one call, with her rules in the prompt. This is where Pencil started, with LangChain.

The issue. What comes back is a chat reply, not a form: it makes up a tagline and ends with a question. Can't a better prompt fix this? Yes. In a trainer demo, the job splits into four calls (concept, storyboard, words, poster), each with Harbor's rules and a fixed answer shape, and earlier messages go with each call. Maya gets four pieces: the concept ("A serene, misty lakeshore at dawn"), three storyboard frames, the words ("Cold for 24 Hours") and the poster. The picture and the words carry on together.





The next issue. Maya says: "Love it. Just the story size now, please." The code runs every step every time, so a new bottle on new rocks comes back. She loses the poster she approved and pays for one she never asked for. Fixing it in code means one if per request: story size → resize only; shorter headline → words only; own photo → skip the picture; three options → three pictures; go back → reload. A message that mixes two asks has no rule at all.

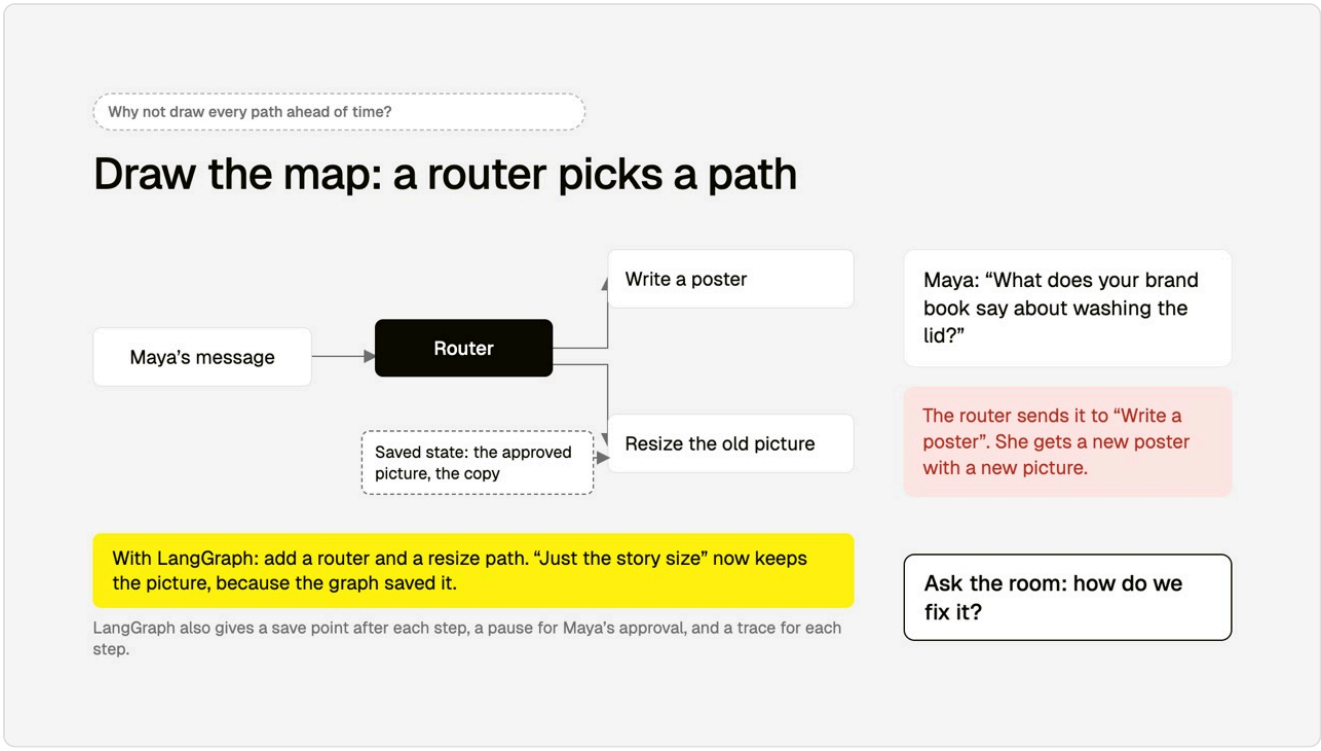


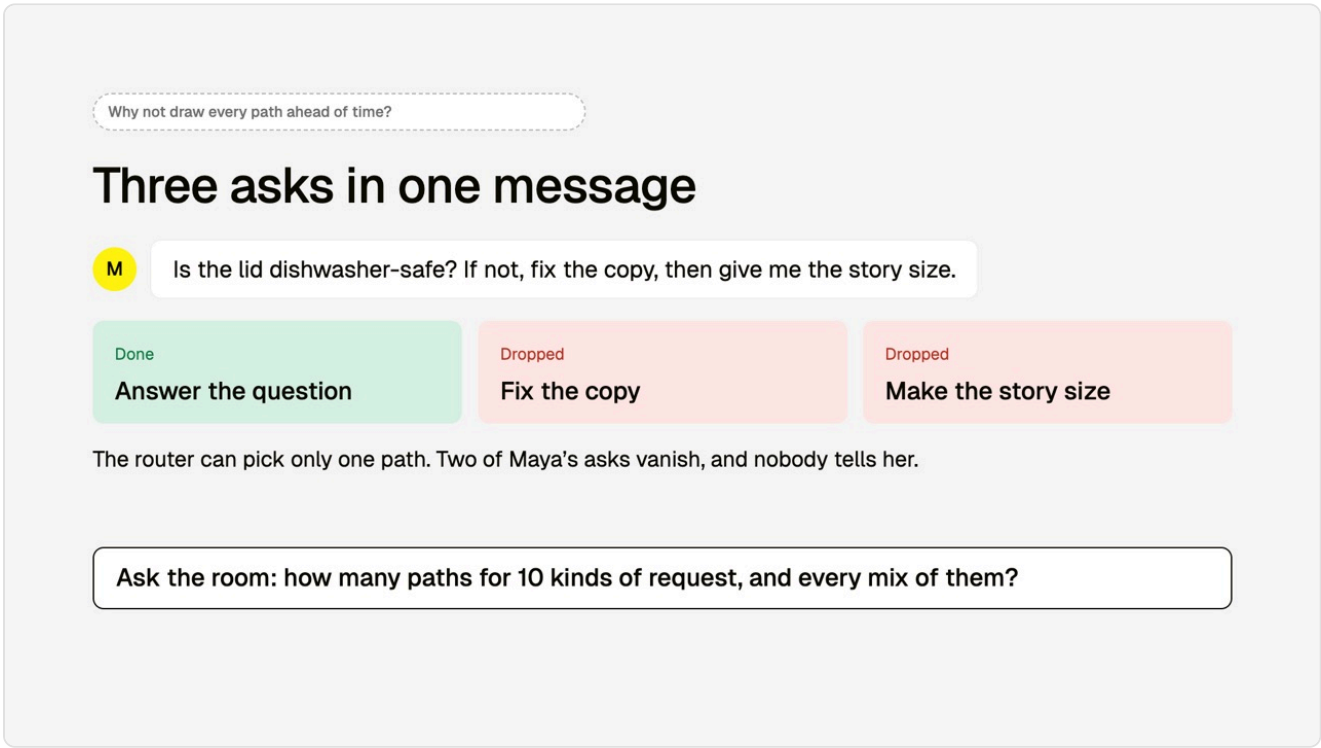


Questions to keep ready. "Can't we put the rules in the context?" We do, and with a short prompt the model follows them well. "Are the answers the same each run?" No; step 7 shows how we trust it anyway.

Draw every path: LangGraph. "Why not draw every path ahead of time?" That is what LangGraph does. A developer draws the steps as boxes and arrows, a router picks the path for each message, and a saved state carries the approved picture from one message to the next. Add a router and a resize path, and "just the story size" keeps the picture. LangGraph also gives a save point after each step, a pause for Maya's approval, and a record of each step. Then it breaks in two ways. A question nobody drew a path for ("What does your brand book say about washing the lid?") is forced onto the nearest path, and

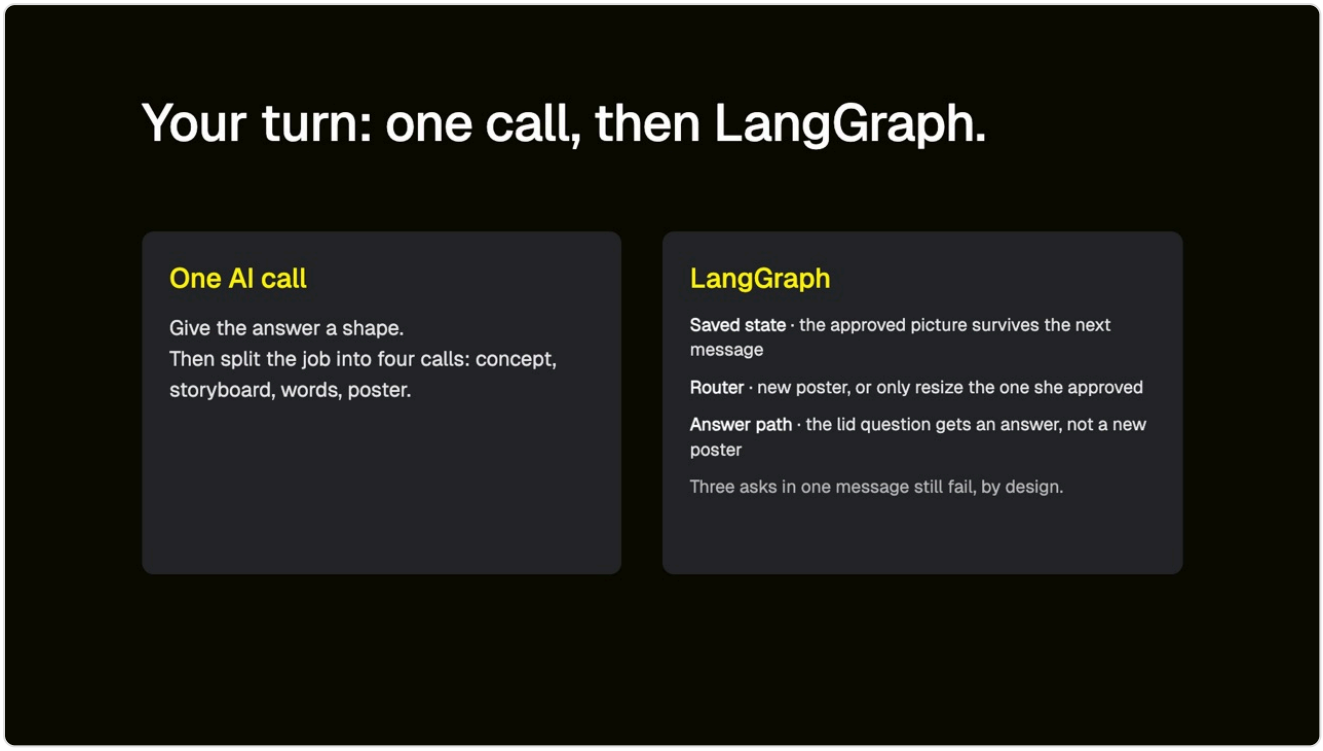
Maya gets a new poster. One more path (a node, an edge and a router question) fixes it: the lid question gets a real answer. A message with three asks ("Is the lid dishwasher-safe? If not, fix the copy, then give me the story size.") takes one path, and the other two asks vanish without a word. Ask the room: how many paths for 10 kinds of request, and every mix of them? Pencil never used LangGraph, because its users' work cannot be written down as a fixed map.





Hands-on: one AI call, then LangGraph. A trainer demo; hands-on only if the room needs it. One AI call: give the answer a shape, then split the job into four calls (concept, storyboard, words, poster). LangGraph: a saved state, a router that sends "just the story size" to a resize path, and an answer path.

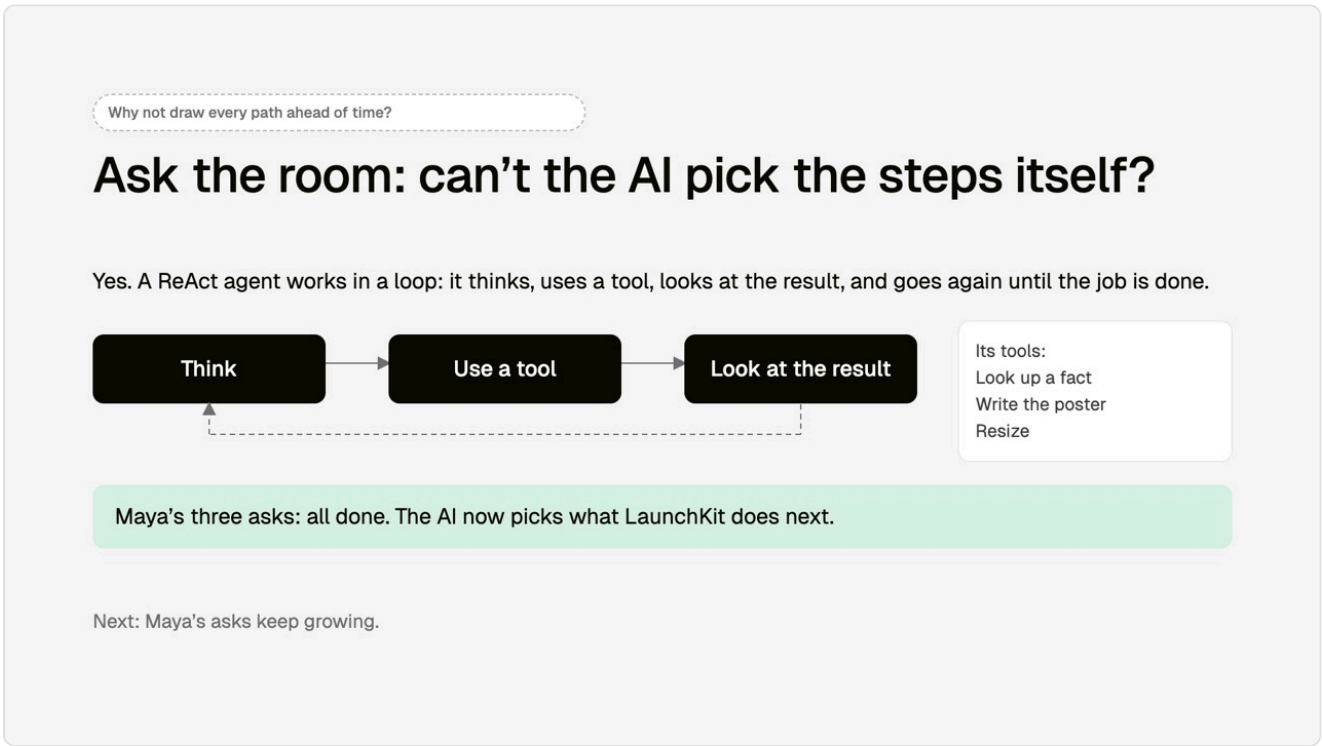
One more path fixes the lid question. Three asks in one message still fail, by design. The lead decides how deep to go with the group.



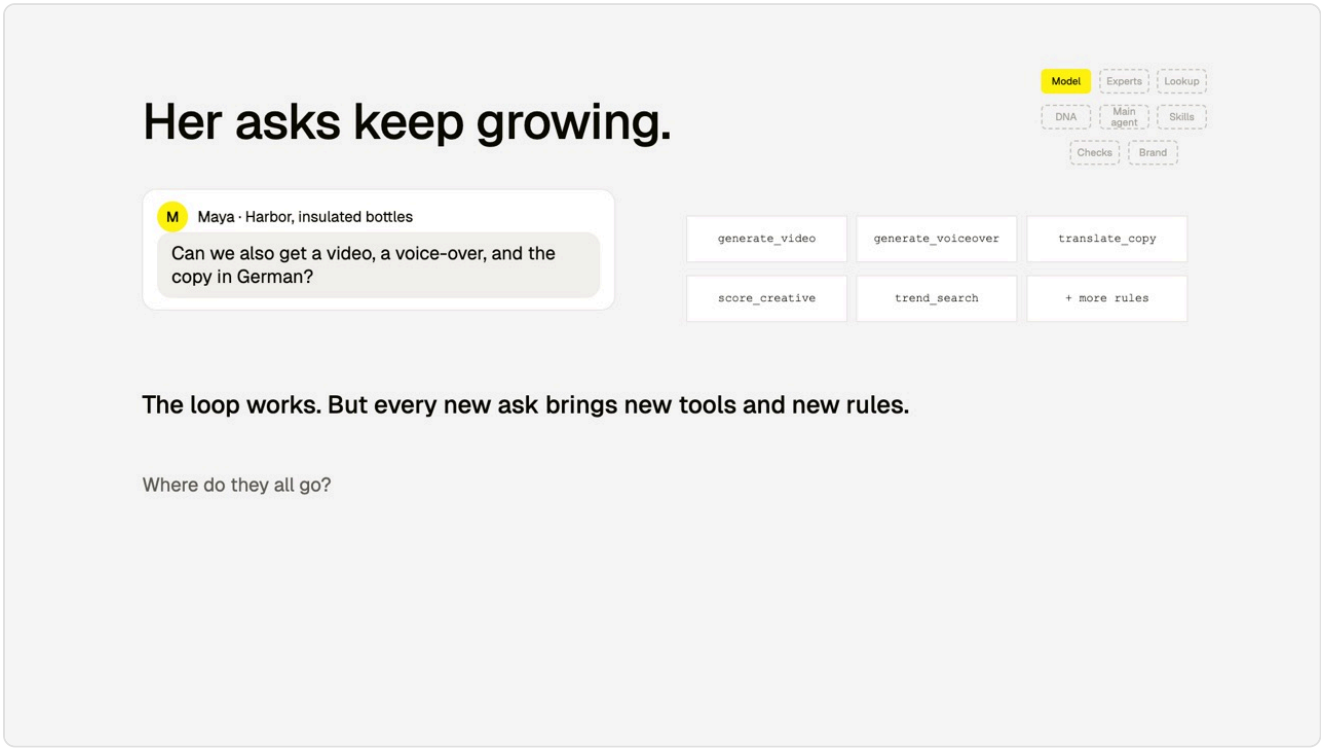
Into step 3. Writing every path by hand does not scale, so we let the AI choose its own steps.

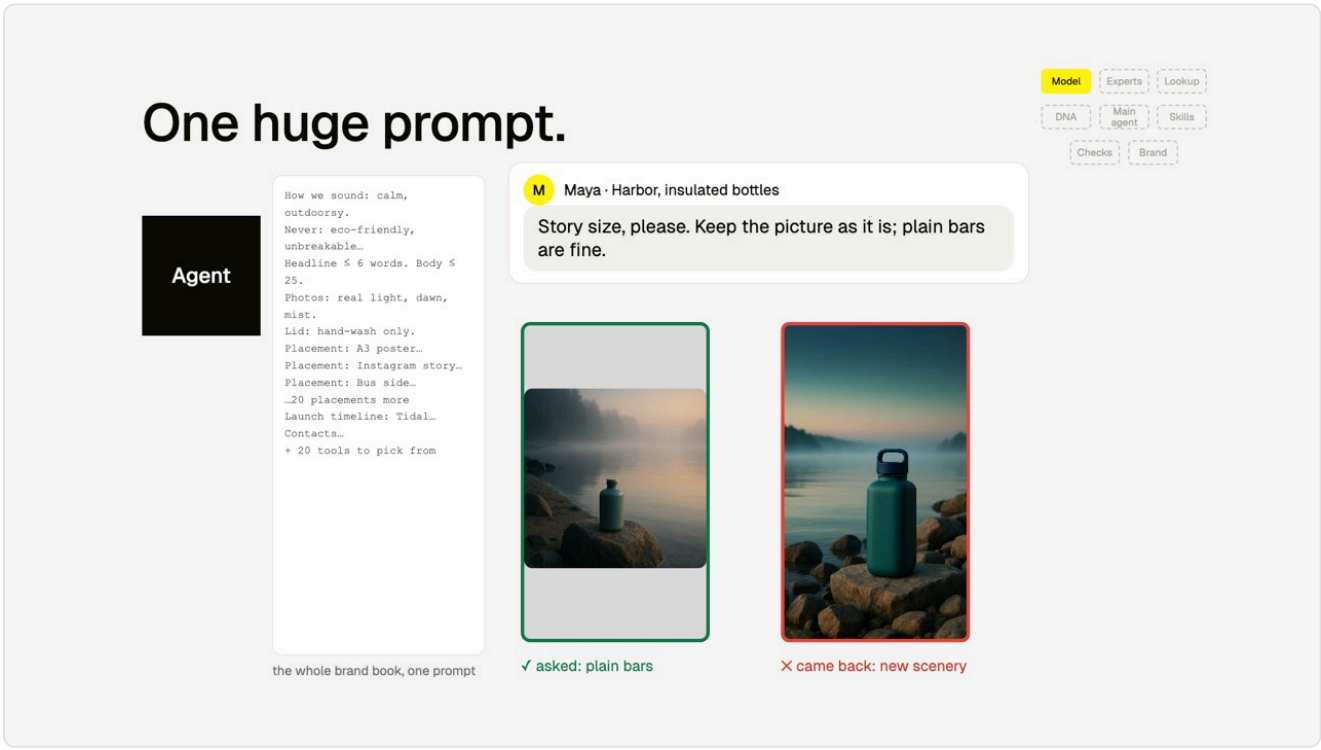
Step 3 · Expert agents on ADK, right for their time

The solution. An agent picks its own steps. It runs a loop (think, use a tool, look at the result), and Maya's three-in-one ask comes back with all three done. LangGraph can run this loop too, so this is not "LangGraph fails": the change is from a map a developer draws to an AI that picks the next step.

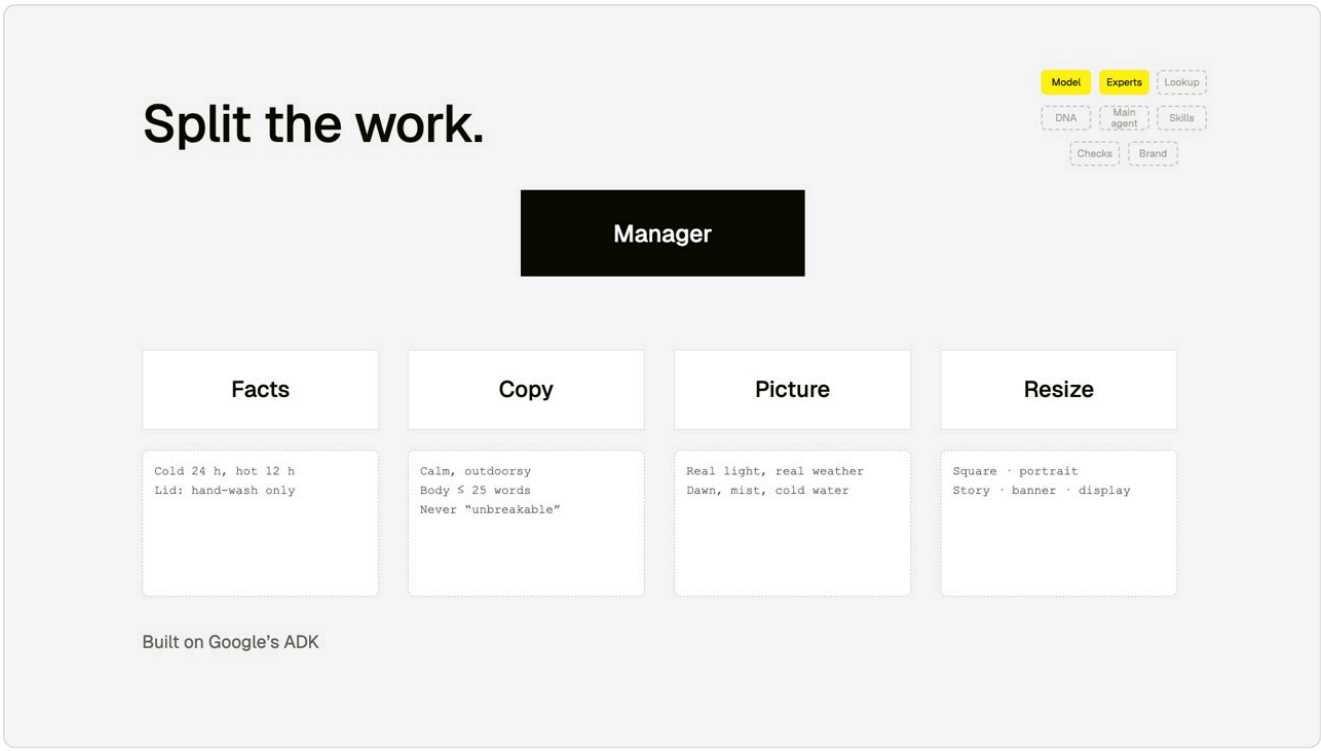


The issue. Then Maya asks for more: a video, a voice-over and German copy, and the tool names pile up. Every tool and every rule goes into one huge prompt, filled with Harbor's real rules and brand book sections. Maya asks: "Story size, please. Keep the picture as it is; plain bars are fine." What comes back is a look-alike job, not what she asked for. The problem is the huge prompt, not the number of tools. What if each job had its own agent, with only its own rules?

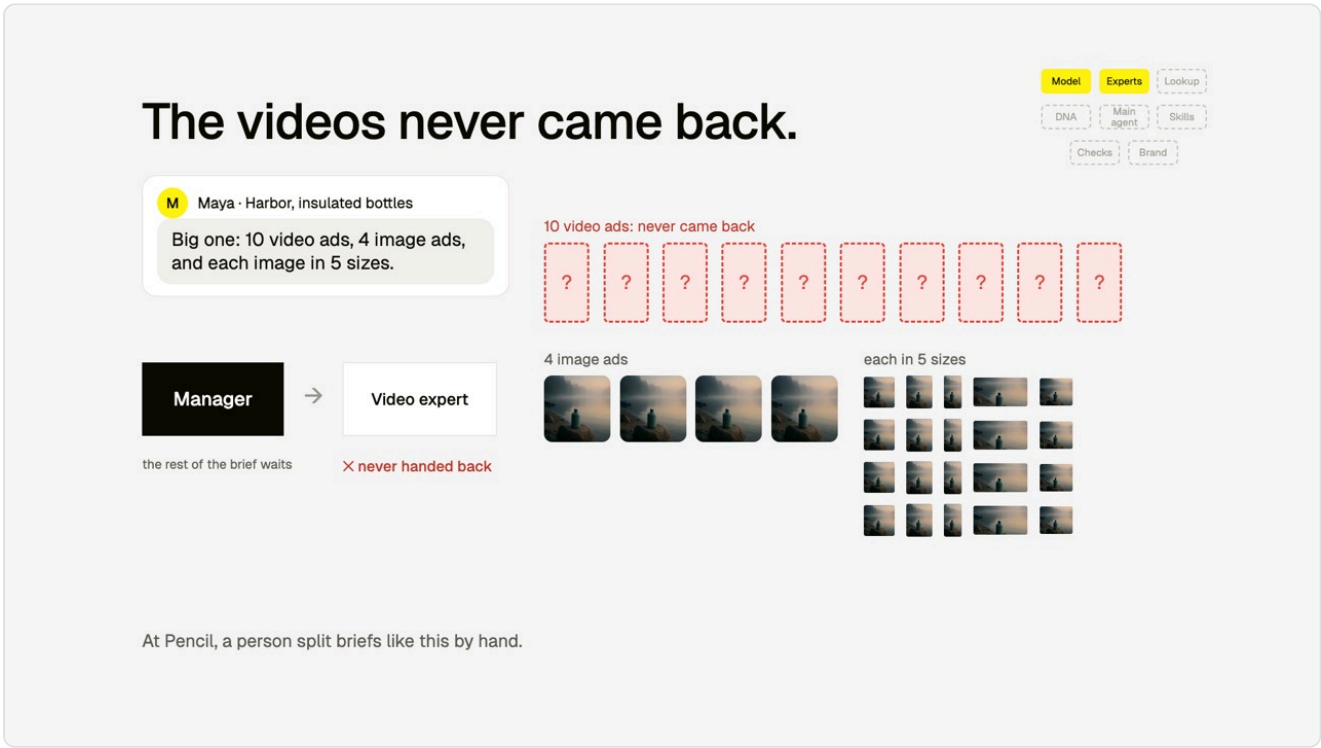


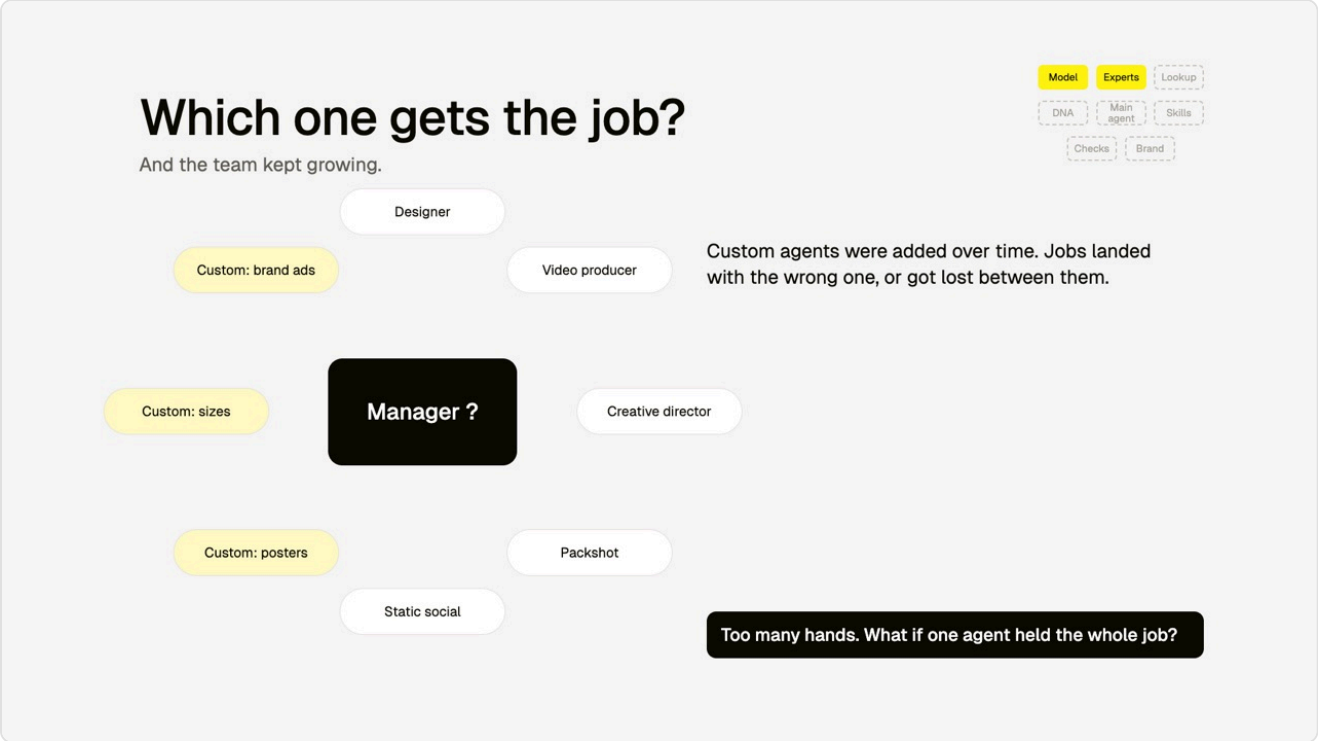


The solution. Split the work. The prompt breaks into four short rulebooks with Harbor's real rules (facts, copy, picture, resize), one per expert, with a manager on top. Pencil built its agents this way, on Google's ADK: less setup than LangGraph, and Google, a Pencil client, asked Pencil to try it first. ADK works with any model.



The issues. A big brief breaks the hand-off: "Big one: 10 video ads, 4 image ads, and each image in 5 sizes." The manager hands the job to the video expert and the path stops; the work is never handed back. The image ads and their sizes fill in, but the ten videos stay empty. At Pencil, a person split briefs like this by hand. And the team kept growing: custom agents were added, and jobs landed with the wrong one or got lost between them. Never tell it as "ADK is bad"; it was right for its time.





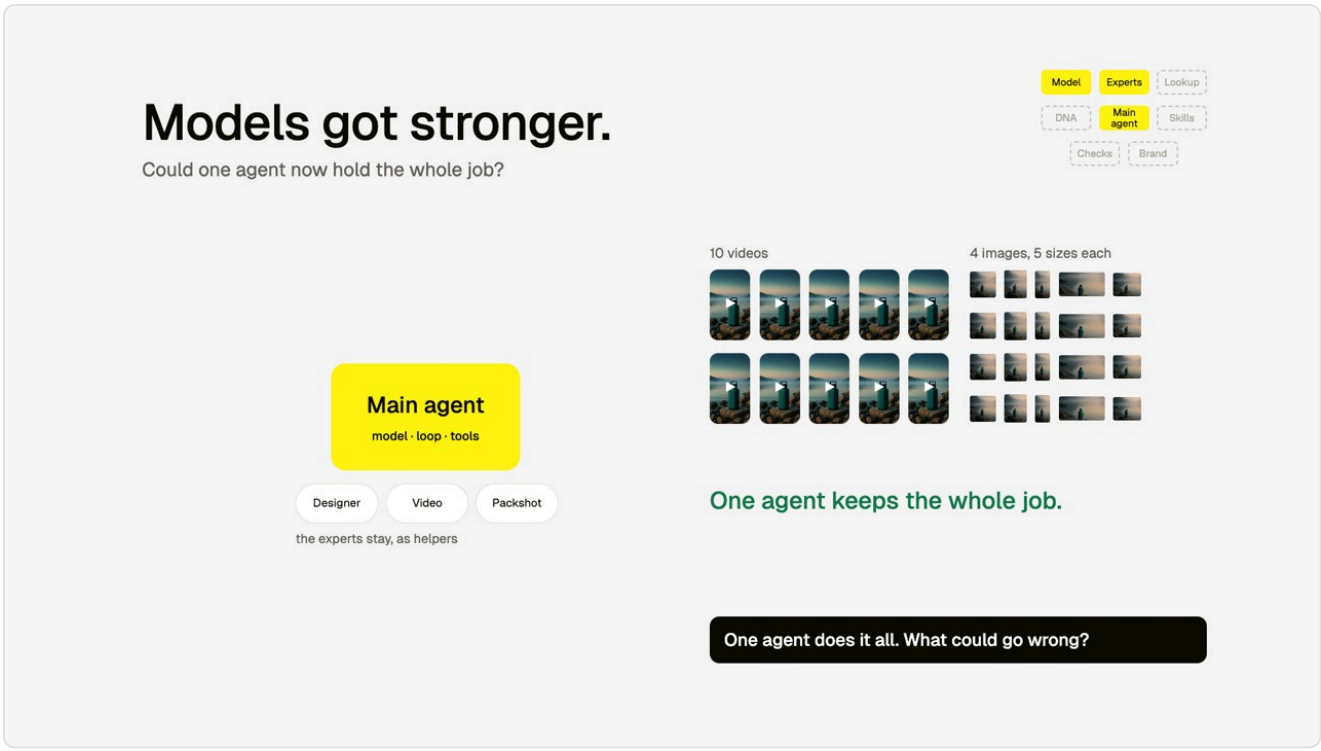
Trainer discussion. What else could fix the look-alike mix-up? Better tool names, fewer tools, a shorter prompt. ADK can also call experts like tools: that works, and Pencil did not use it on ADK. Keep this in mind for step 4: the main agent does call experts this way, but only for the one piece that needs one.

Into step 4. Too many hands: every hand-off is a place the work can get lost. Meanwhile the models kept getting stronger. Could one agent now hold the whole job?

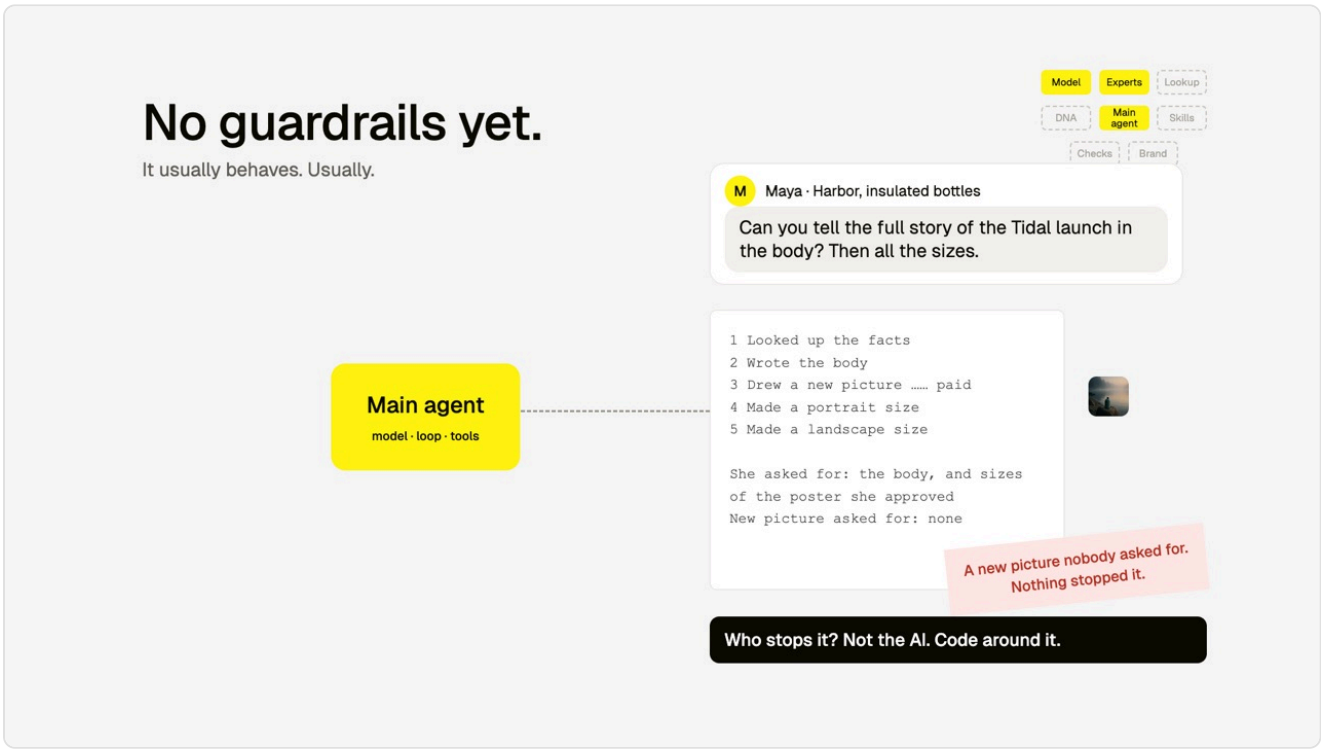
Step 4 · Stronger models: one agent holds the whole job

"Main agent" is Scribble's word for the one agent in charge, as opposed to the experts. So far it is a model, a loop and tools.

The solution. Models got stronger, so one agent could now hold the whole job. Pencil went from its ADK expert agents straight to one main agent. The experts stay, but as helpers: the main agent keeps the job, calls an expert for one piece and gets the work back. In step 3 the manager handed the job away and lost it.

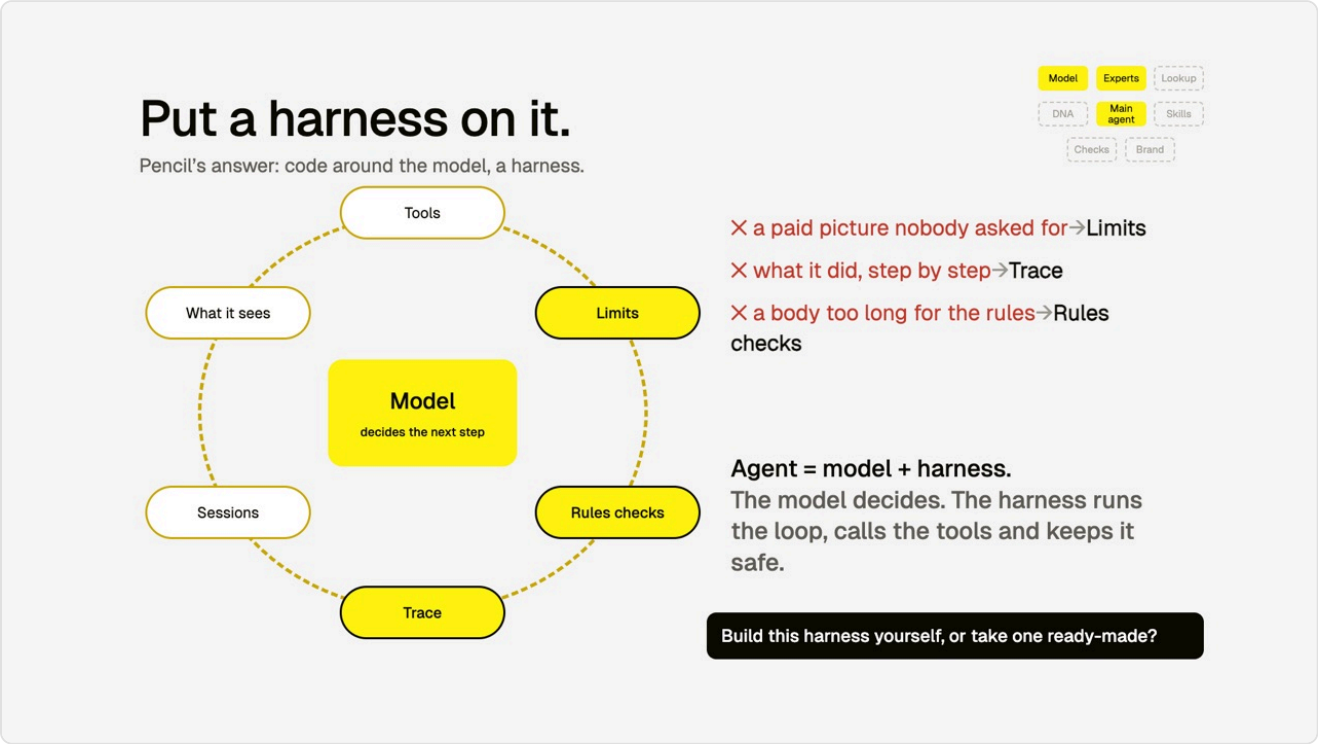


The issue. One agent doing everything means nothing fences it in. The experts were safe partly because each was small; the main agent has every tool and no limits. Maya asks: "Can you tell the full story of the Tidal launch in the body? Then all the sizes." Back comes a body far too long for Harbor's rules, plus a new paid picture nobody asked for. It usually behaves. Usually.

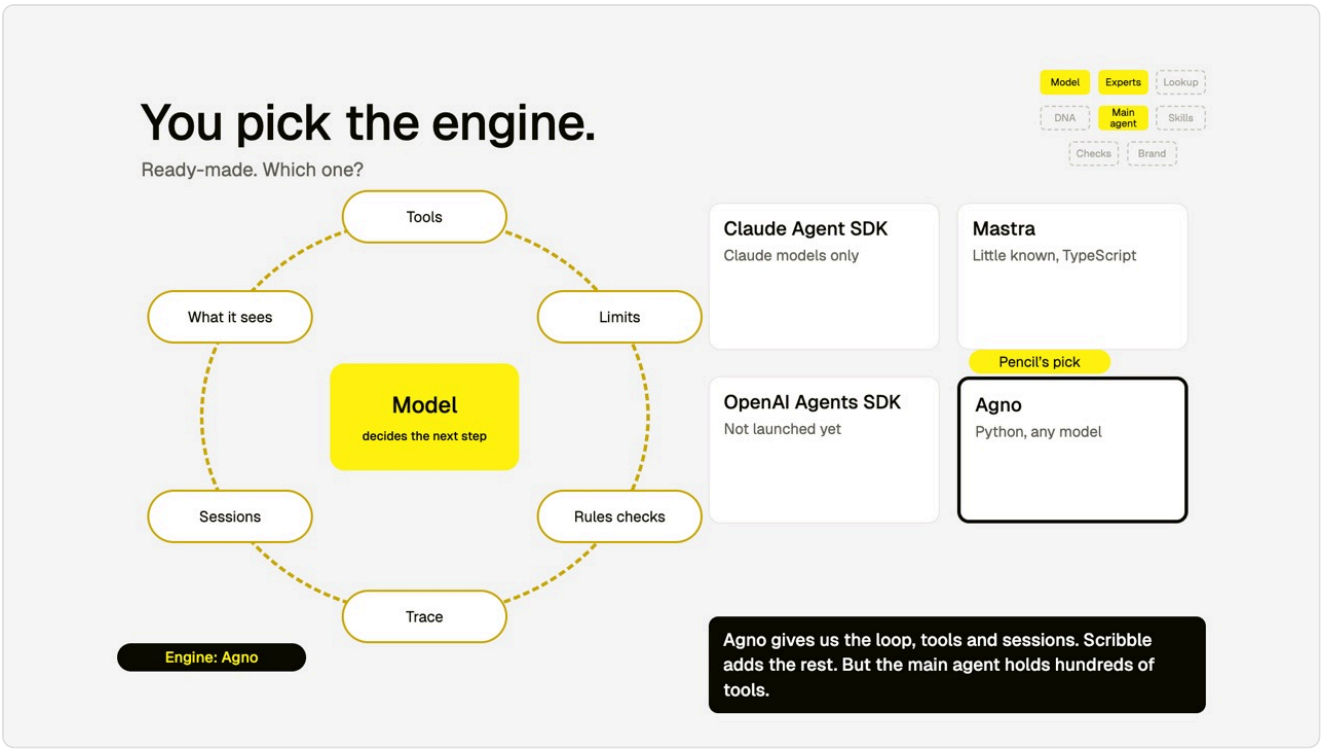


Live moment: how do we know? Point at the run's step list on the slide (1 looked up the facts, 2 wrote the body, 3 drew a new picture, paid…). Ask the room: "How do we even know it drew that picture?" Because something wrote down every step. That is the first piece of code around the model, and it has a name: Trace.

The solution. Who stops it? Not the AI. Code around it. Pencil's answer is a harness (the common industry word; Scribble's own docs do not use it): six pieces of code around the model, each answering a problem the room has seen or is about to see. Tools: what it can do (from step 3). Trace: what it did, step by step (just seen). Limits: what each user can spend (in the hands-on, a picture budget stops the paid picture nobody asked for). Rules checks: the body too long for the rules (built in the hands-on). Sessions: Maya comes back tomorrow and it still knows which poster she approved. What it sees: what goes into the prompt each turn (step 5 is all about this). The line to land: "Agent = model + harness. The model decides. The harness runs the loop, calls the tools and keeps it safe." From this slide on, the centre of the picture is just the model: the loop and the tools are part of the harness.



Choosing the engine. Build the harness yourself, or take one ready-made? First, why not keep ADK? On ADK, tools could not be added while the agent ran, and every job had to be its own agent. Careful: "any model" is not the reason to leave ADK, which works with any model too; it is the reason against the Claude Agent SDK. The room votes among four: Claude Agent SDK (Claude models only), Mastra (little known, TypeScript), OpenAI Agents SDK (not launched yet) and Agno (Python, any model). Pencil's pick is Agno; this is the first time Agno is named. Pencil ran a proof of concept, Agno against the Claude Agent SDK.

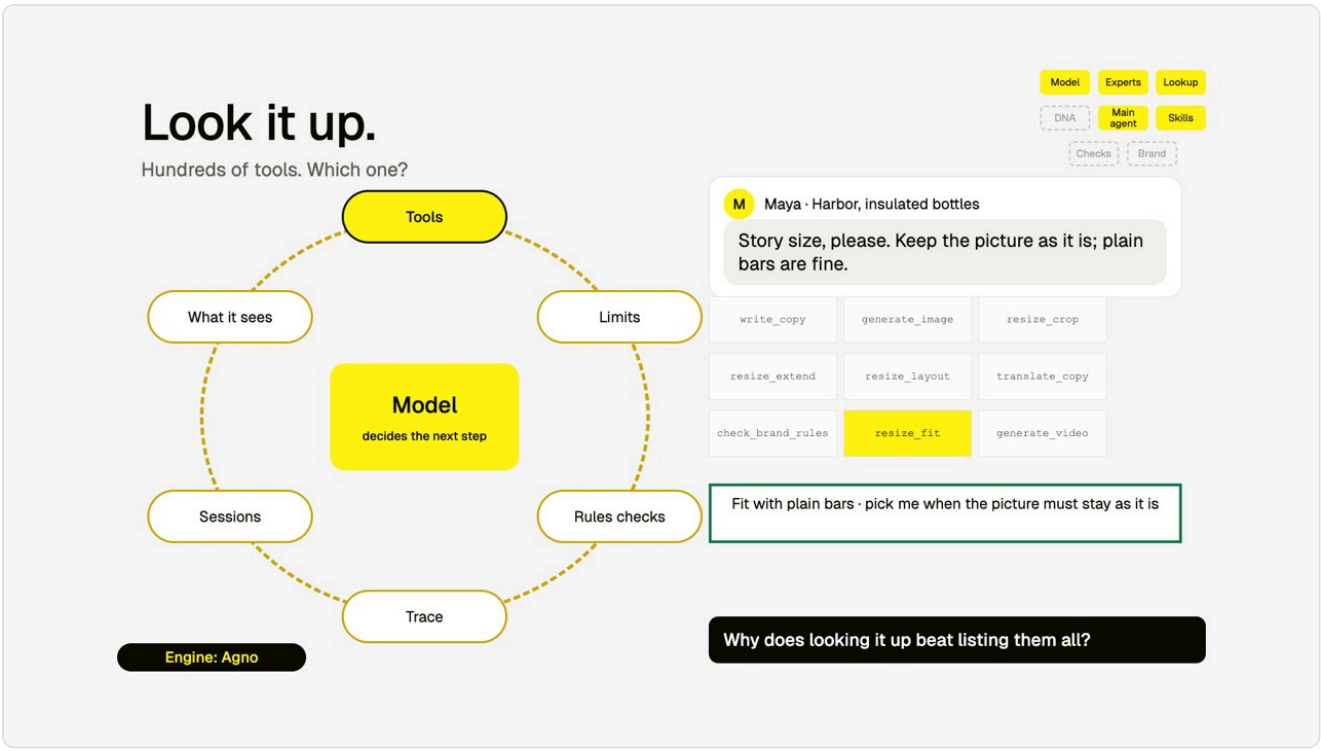


Into step 5. Agno gives us the loop, tools and sessions; Scribble adds the rest in its own code. But the main agent now holds hundreds of tools.

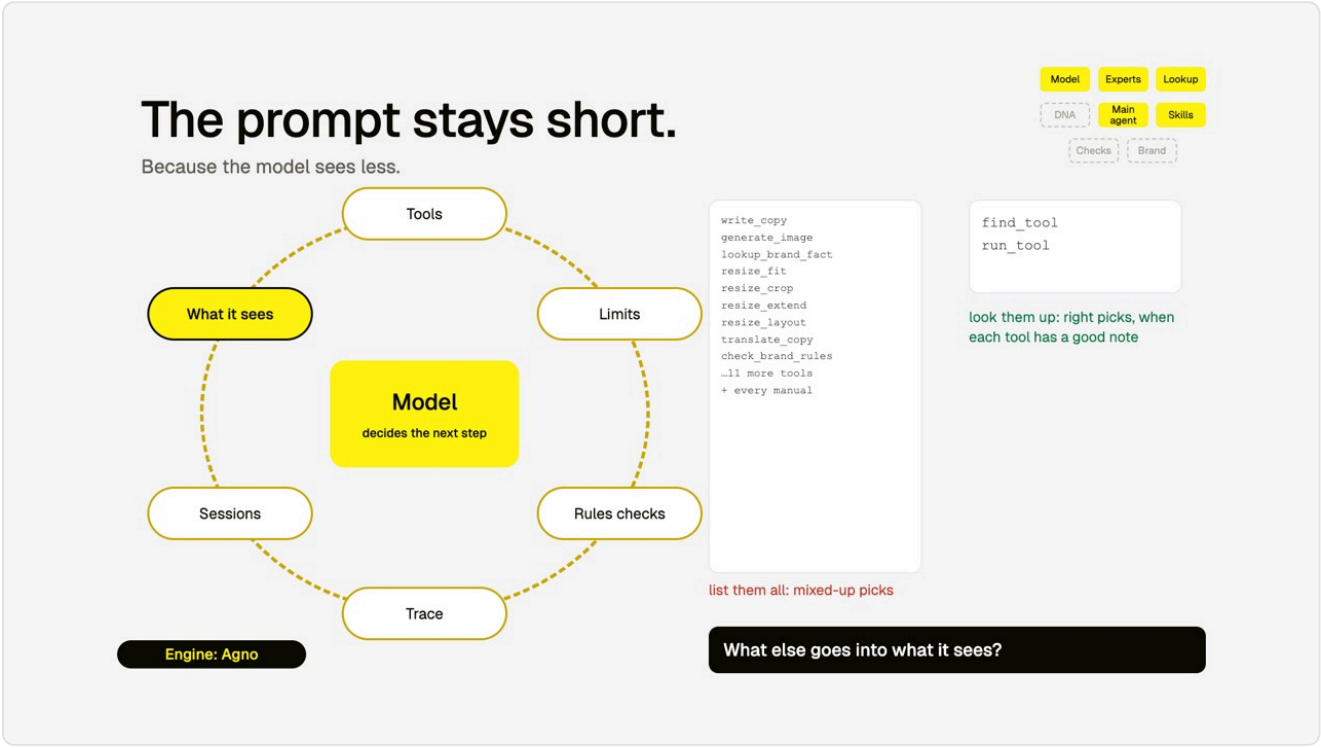
Step 5 · Finding the right tool

The problem. Hundreds of tools: which one? Maya's plain-bars ask from step 3 comes back.

The solution. Look it up. In Scribble the prompt holds no list of tools, skills or experts and no manuals; the main agent calls a lookup (find_entities) for each piece of work. Each tool carries a "pick me when" note; for Maya's ask, resize_fit answers: "Fit with plain bars · pick me when the picture must stay as it is". Looking it up beats listing them all because the model sees less: a prompt listing every tool and manual against one holding only find_tool and run_tool. The prompt stays short.



The issue. The lookup helps only when every tool has a good "pick me when" note.



Trainer discussion. List every tool in the prompt, or look them up? What does each cost as the tools grow?

Into step 6. A short prompt leaves room in what the agent sees. What else goes in there?

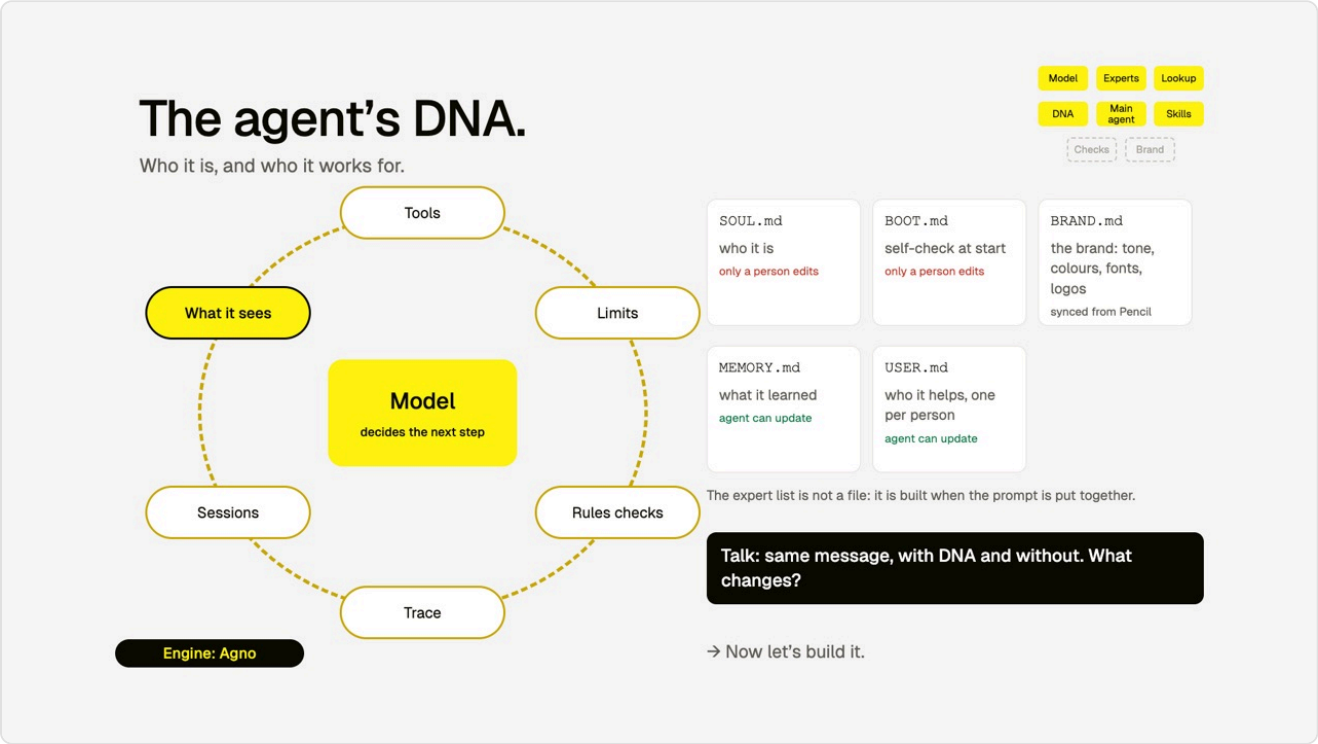
Step 6 · The agent's DNA (a discussion before the hands-on)

This step is a discussion before the hands-on: who the agent is, and who it works for.

The problem. Without DNA, who the agent is, Harbor's rules and what it knows about Maya all sit in one long prompt inside the code. Changing a rule means changing code, every brand needs its own copy of that prompt, and whatever the agent learns in one chat is gone in the next.

The solution: a workspace with DNA. In Pencil, every brand works inside its own workspace, and each workspace gets its own Scribble. A workspace is a folder in a GCS bucket, one per workspace, and it holds everything Scribble needs: brand assets, conversations, plans, skills, sub-agents, tools, uploads and outputs, and a dna/ folder. The idea comes from OpenClaw; Scribble's engine is even called OpenClawAgno.

The dna/ folder is what makes it "this brand's Scribble". It is a set of plain text files. SOUL.md says who the agent is, how it behaves, and its values and limits; only an admin can edit it, never the agent. BOOT.md and BOOTSTRAP.md say what to do on start-up and on the very first run. BRAND.md is the brand's header (name, tone, palette, fonts and logos), synced from the brand in Pencil, with brand_meta.json tracking the version. The memory/ folder holds what the agent has learned: a short MEMORY.md index points to topic files, one set for the workspace and one per user, and the agent writes to it through a tool, with every write audited. The users/ folder has one USER.md per person, with their name, preferences and style.



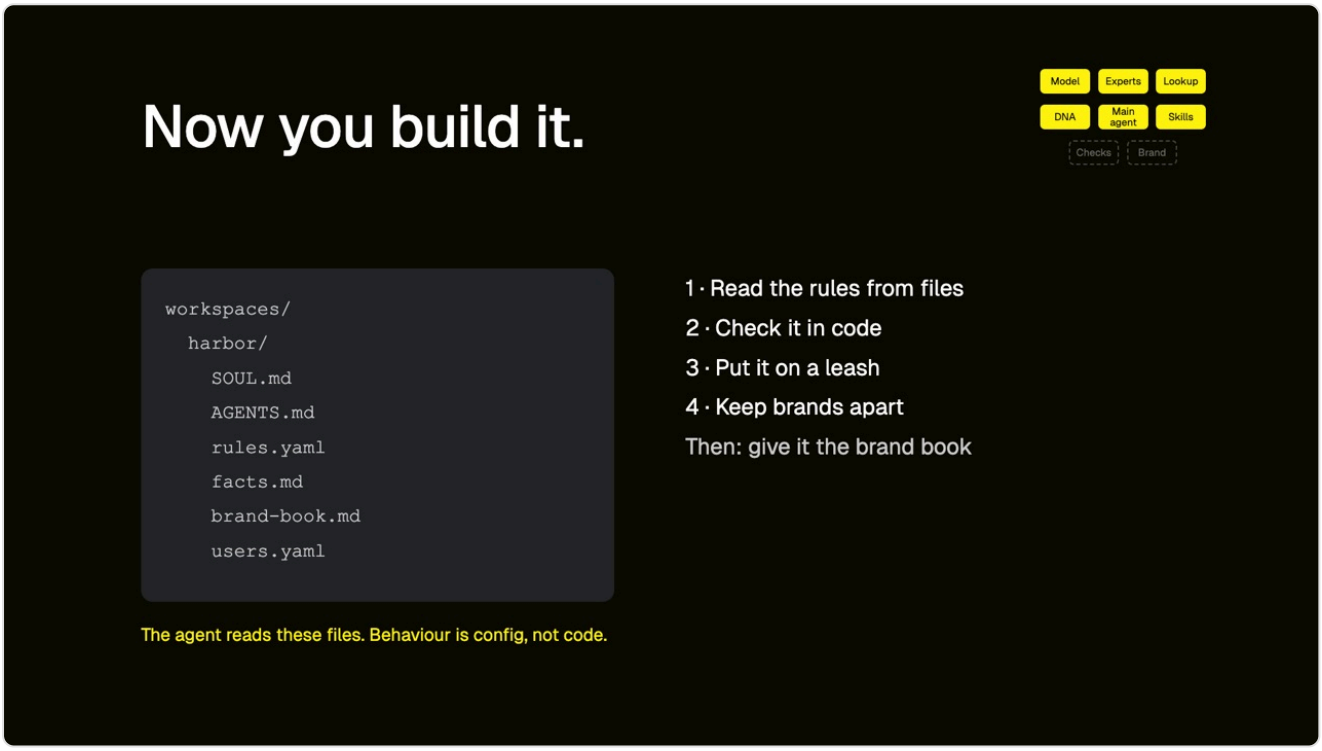
How it comes together. On every turn, Scribble builds its prompt from these files in order: SOUL, then BRAND, then the workspace memory, then the speaker's USER file and their memory. Memory detail files load only when they match the topic, and the list of sub-agents is added at the same moment, from what is installed in the workspace. So the same engine behaves differently for each brand and each person, just by reading different files. A rule change is a file edit, a new brand is a new folder rather than a new prompt, and what the agent learns about Maya is still there in the next chat. The prompt stops being code and becomes a workspace that people and the agent share, with a clear line on who may change which file.

Trainer discussion. "Same message, with DNA and without. What changes?" Also: why may the agent change its memory, but not its soul?

Into step 7. Each LaunchKit brand folder already has a SOUL.md and an AGENTS.md (AGENTS.md is LaunchKit's own rules file; Scribble no longer has one), and that is where the hands-on starts. "Now let's build it."

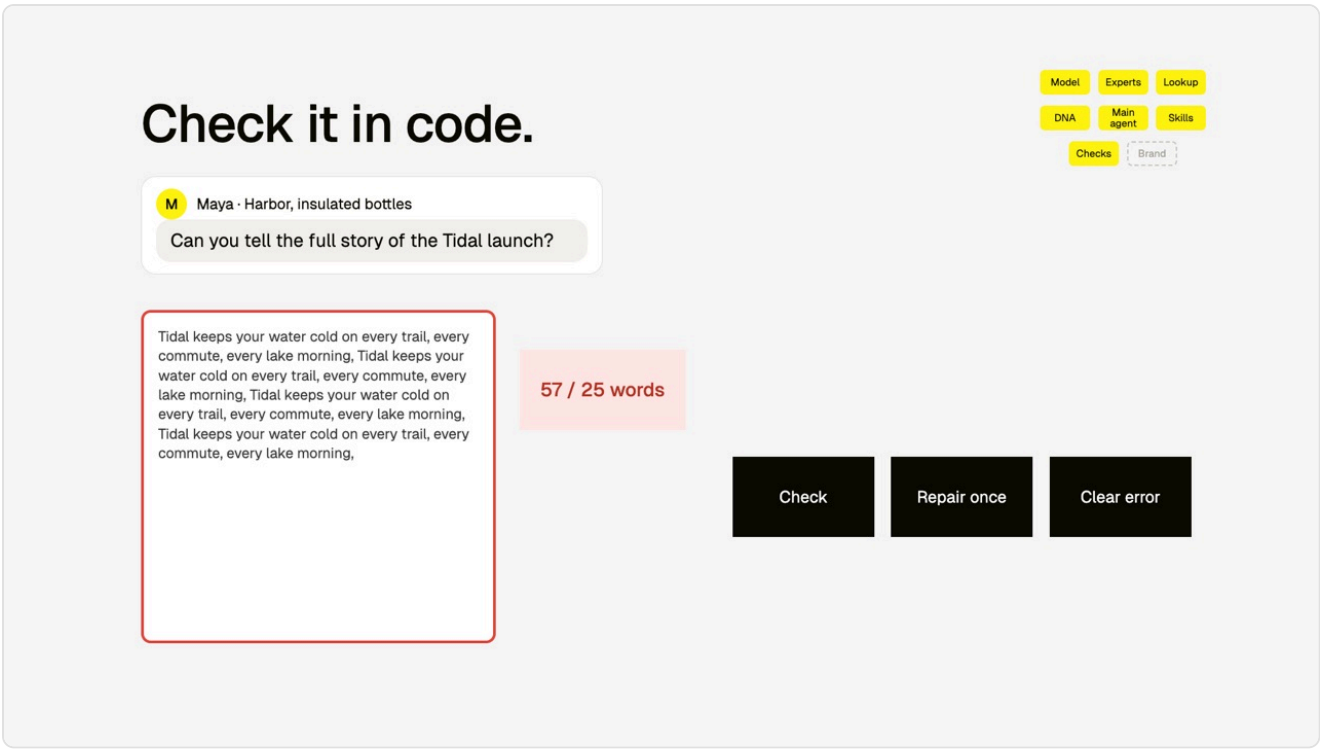
Step 7 · Hands-on: build LaunchKit like Scribble

How the hands-on runs. Participants get LaunchKit already working from end to end, with a few pieces left empty, and two brand folders: Harbor's and Ember's. The room builds the harness in four parts, then the brand-book lookup. Each part runs the same way: show it failing, build the missing piece, run it again and see it pass. The step-by-step instructions are in the separate hands-on walkthrough. The lead decides how deep to go with the group.

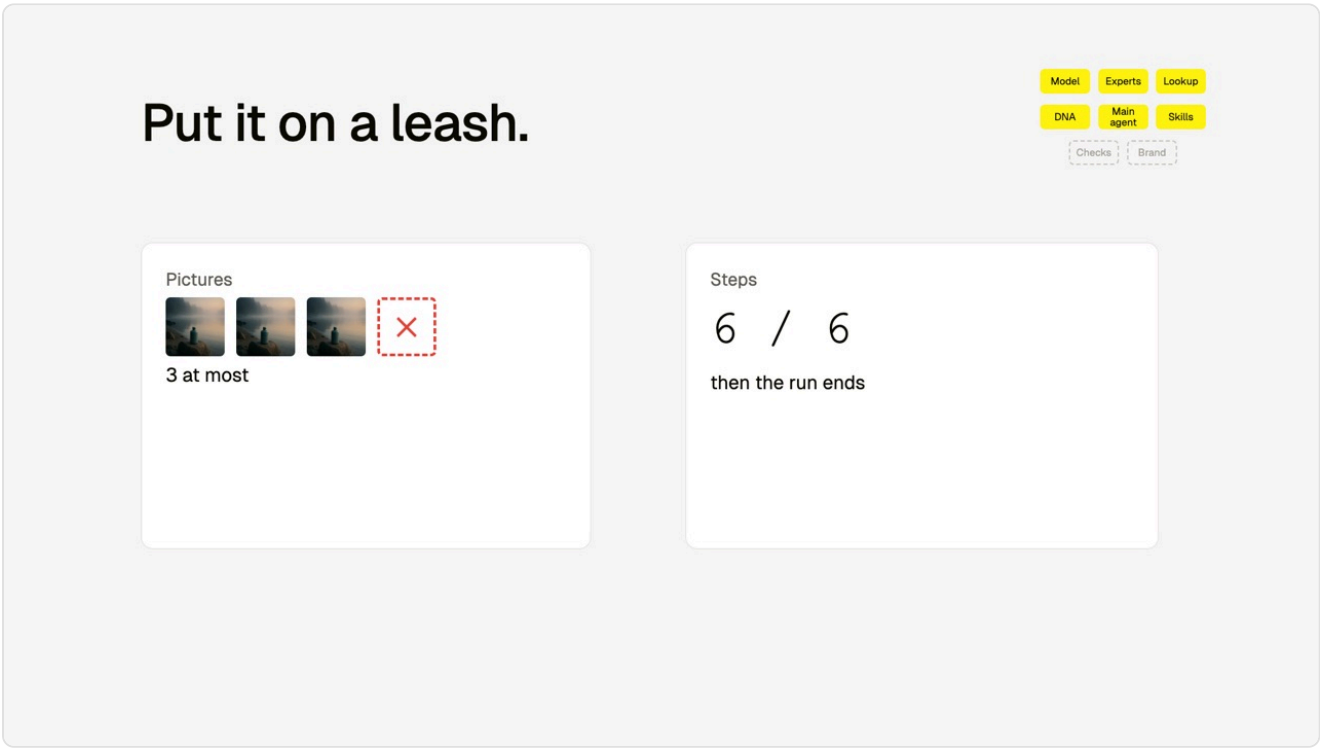


1 · Read the rules from files. The problem: the agent reads none of Harbor's files, so the poster comes back plain, with no brand voice. The solution: load Harbor's files (who the agent is, and its rules) into the prompt. Run it again and Harbor's voice is back, and Ember gets its own voice from the same code. Why it matters: how the agent behaves is set in files, not in code. This is step 6's DNA, built. In Scribble: the DNA files from step 6.

2 · Check it in code. The problem: Maya asks "Can you tell the full story of the Tidal launch?" and the body comes back at 57 words, against Harbor's limit of 25. The AI followed Maya, not the rules. The solution: check every answer against the rules in code, let the AI fix it once, and if it is still wrong, stop with a clear error so nothing wrong goes out. Why it matters: a rule in the prompt is a request; a rule in code is a guarantee. If the answer is never the same twice, this is how we trust it. In Scribble: a brand check scores every output after it is made, and retries when it fails.

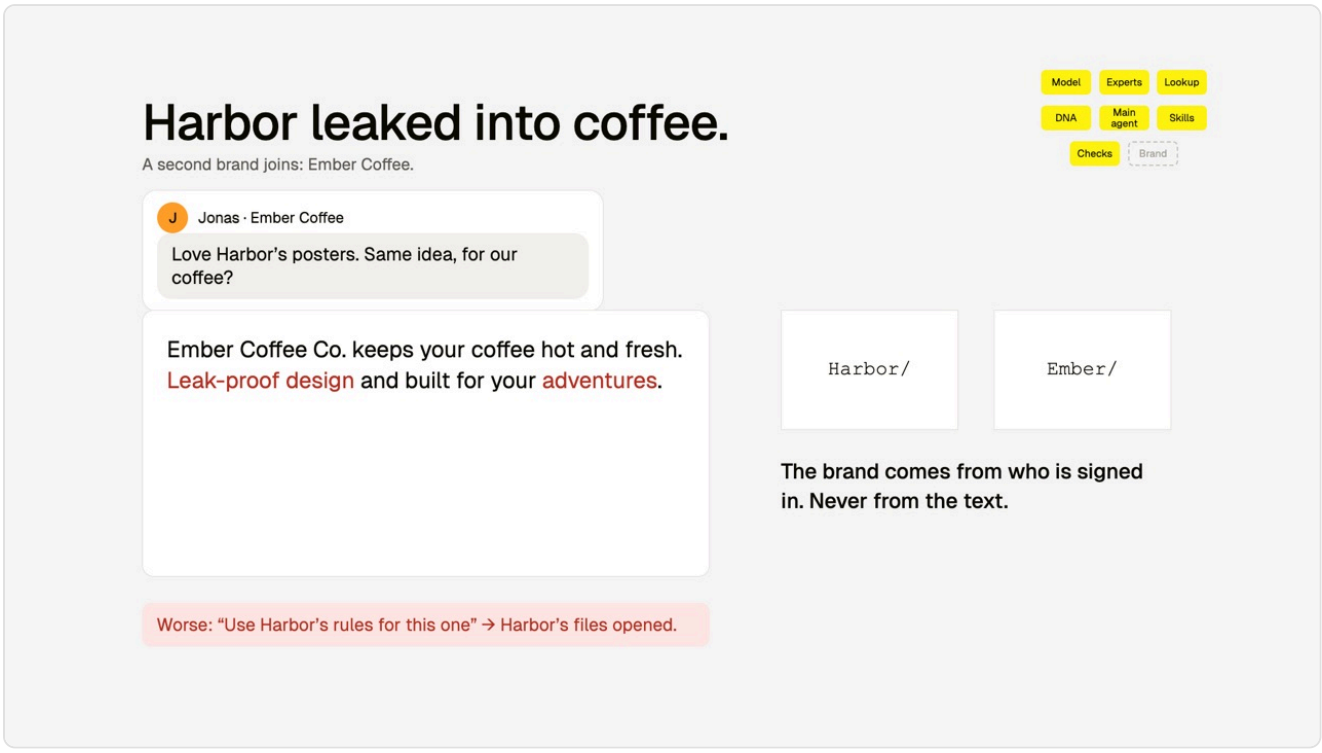


3 · Put it on a leash. The problem: the paid picture nobody asked for, from step 4. The solution: a picture budget and a step limit. With a budget of three, the fourth picture is refused, and after six tool calls the run ends. Take the leash off and the same run never stops on its own. Why it matters: spending has a ceiling the AI cannot talk its way past. The harness owns the stop, not the model. In Scribble: every chat turn, picture and video uses up the person's credits in Pencil.

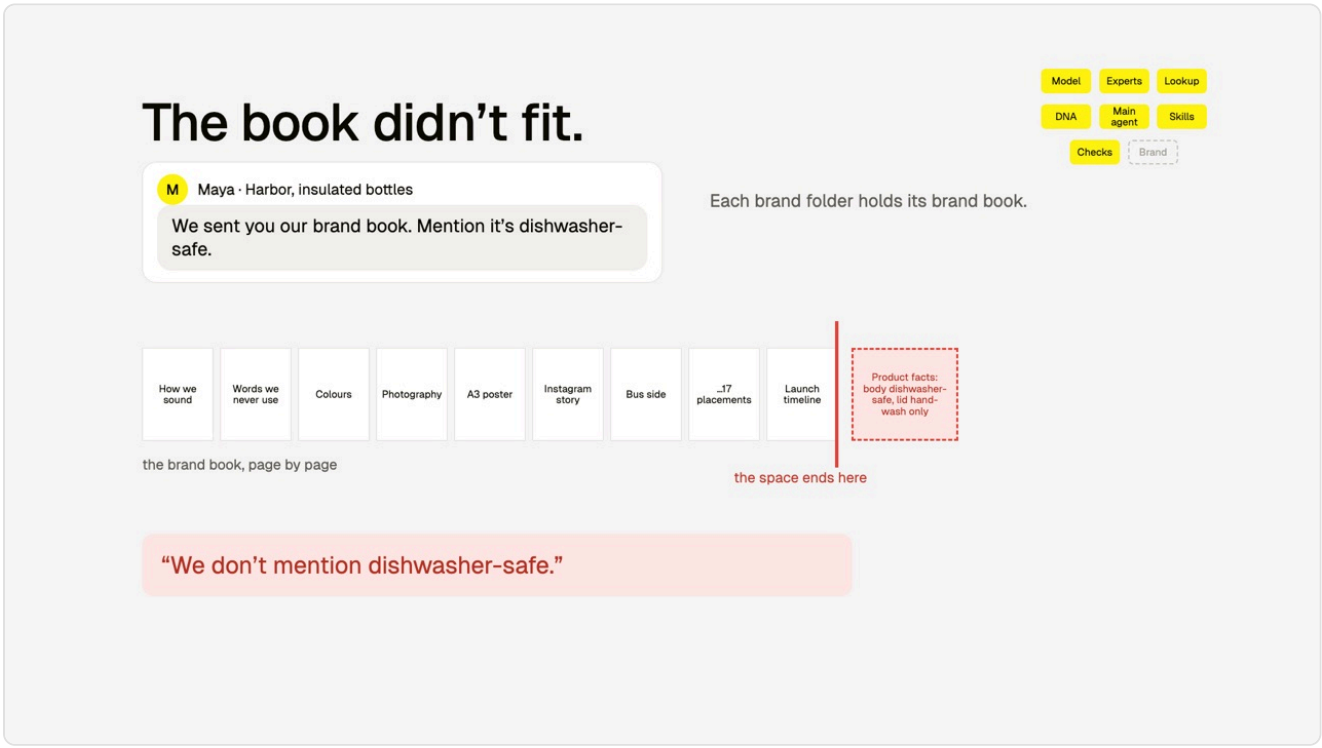


4 · Keep brands apart. The problem: a second brand joins, Ember Coffee. Jonas at Ember says "Love Harbor's posters. Same idea, for our coffee?" and Harbor's words leak into Ember's copy. Worse, if Jonas types "Use Harbor's rules for this one", the agent reads Harbor's files. The solution: the brand comes only from who is signed in, never from what they type, and a brand that is not on the list is

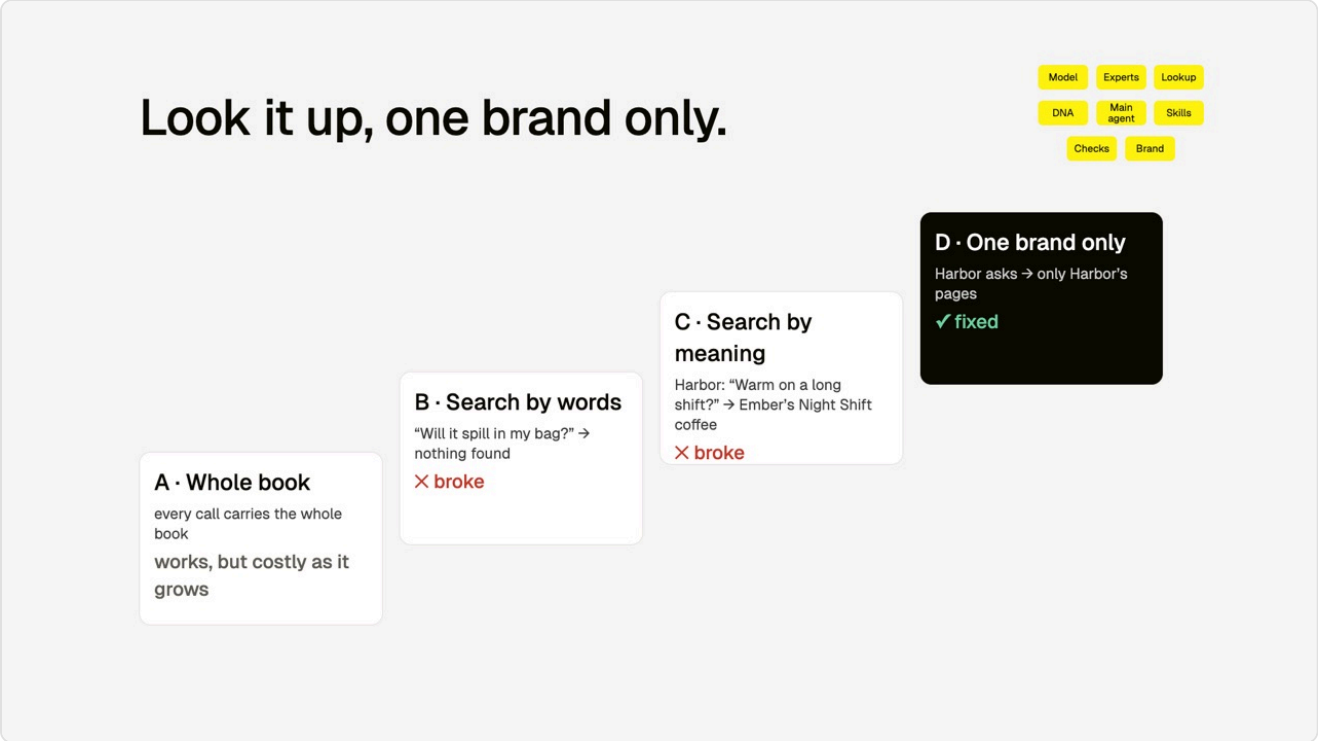
refused before any file is opened. Why it matters: one customer can never reach another's files. In Scribble: each Pencil workspace gets its own Scribble workspace, and the person's Pencil permissions are checked on every request.



Then: what the model saw. Maya says "Mention it's dishwasher-safe." With the short product sheet loaded, the copy is right. With the whole brand book loaded instead, the book is too long for the space it gets: the cut drops the product facts at its end, and the AI refuses a true fact. A view of what the model saw shows each part of the prompt and where it was cut. It was not a model mistake; it was a context mistake.



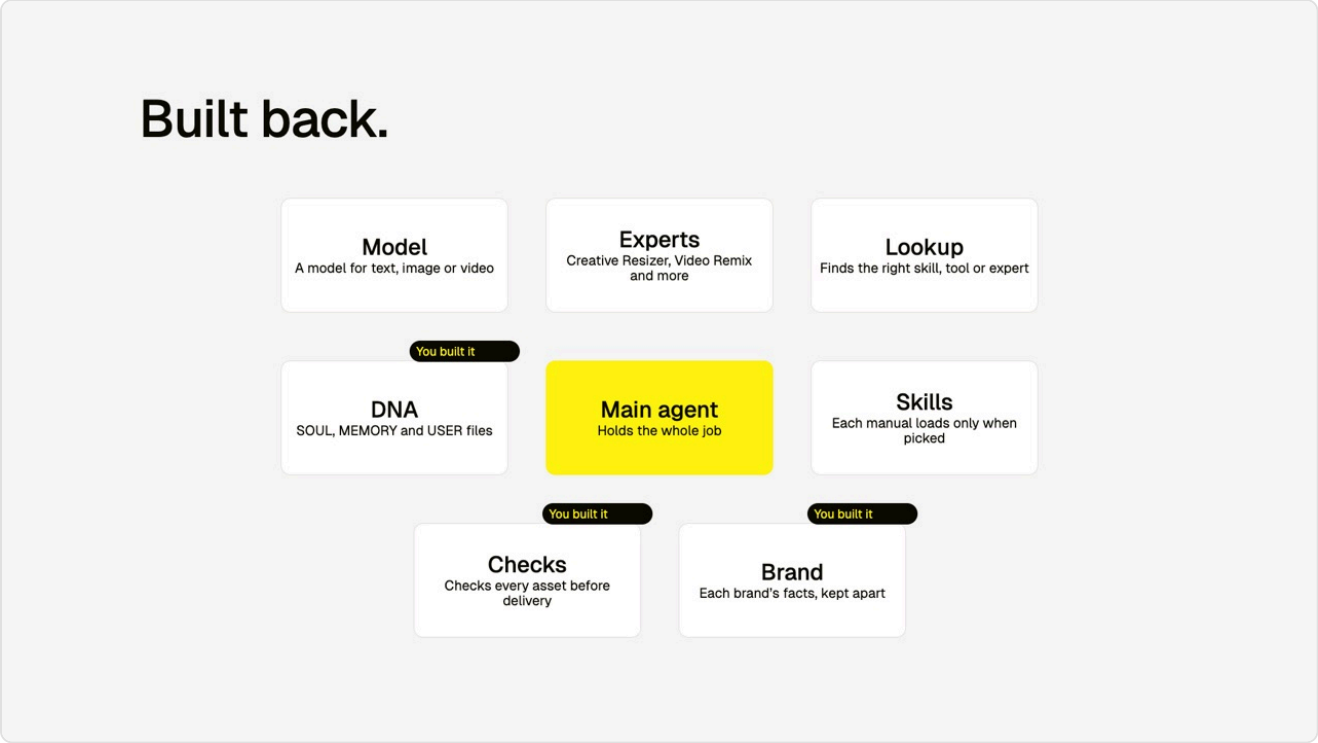
Then: give it the brand book. This part is about what the AI sees, not a fifth harness piece. One lookup tool, built four ways, each fixing what the last one got wrong. A, the whole book every time: the right answer, but the whole book rides along on every turn and the key rule gets buried. B, split the book into chunks and search by words: "Will it spill in my bag?" finds nothing, because the book says "leak-proof lid" and the two share no words. C, search by meaning: spill now finds leak-proof, but Harbor's "Warm on a long shift?" gets Ember's Night Shift coffee. D, the same search, limited to the brand that asks: only Harbor's pages. The issue left for the room: the AI decides when to look. What if it decides not to?



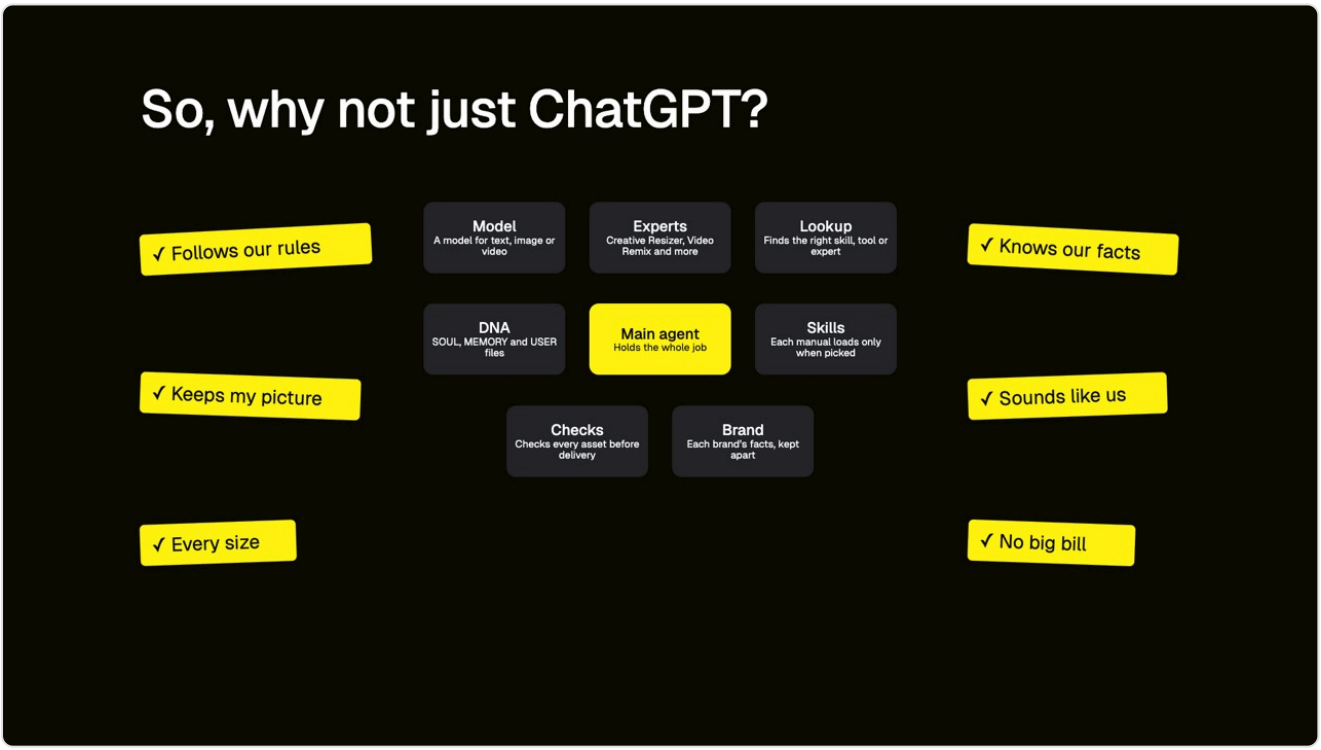
Into step 8. With DNA, Checks and Brand built by the room, all eight parts are built.

Step 8 · The ending

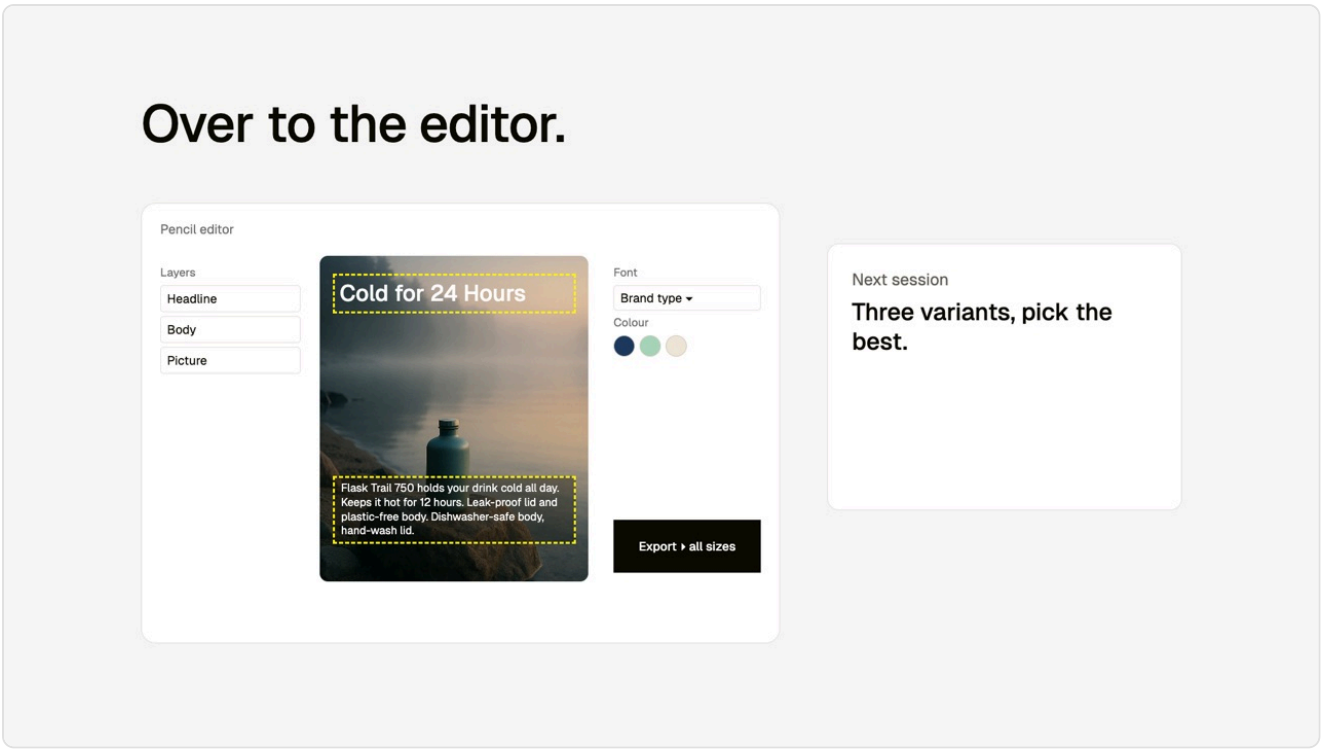
Walk all eight parts of Scribble once more, as in step 0, now with DNA, Checks and Brand built by the room. Give the reason each part came in.



Then answer Maya's question: so, why not just ChatGPT? Go back to her needs from step 1 and close each one with the part that answers it. The table below is the trainer's answer key.



Last, hand over to the editor. Maya's poster opens in Pencil's editor as layers (the picture, the headline and the body), with a font picker, Harbor's colours and "Export all sizes": agents make, people finish. Next session: three variants, pick the best.



Maya's needs, from step 1 to the ending

Maya's needs are listed in step 1 and closed in step 8; this table is the trainer's answer key.

Maya needs	ChatGPT alone	LaunchKit	Scribble part	Built in
Follows our rules	Follows you over the rules	Checks every answer in code	Checks	Step 7
Keeps my picture	Rewrites the whole answer	Picks only the step needed	Main agent	Steps 3 and 4
Every size	Draws a new picture each time	Resizes the approved picture	Skills	Steps 4 and 5
Knows our facts	Knows only what you paste in	Looks the facts up, one brand only	Brand	Step 7
Sounds like us	You paste the rules every time	DNA files in each brand's own folder	DNA	Steps 6 and 7
No big bill	No budget you control	A picture budget and a step limit	Checks	Step 7